

Пермский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Национальный исследовательский университет
«Высшая школа экономики»

Факультет социально-экономических и компьютерных наук

Берсенёв Илья Иванович
**Разработка системы для быстрого декодирования видео в
формате AV1**

*Выпускная квалификационная работа
бакалавра по направлению 09.03.04 Программная инженерия*

Руководитель
Старший преподаватель кафедры
информационных технологий в бизнесе
_____Ланин Вячеслав Владимирович

Пермь 2026

АННОТАЦИЯ

Берсенёв Илья Иванович, Разработка системы для быстрого декодирования видео в формате AV1: исследовательская работа бакалавра 09.03.04 программная инженерия / Берсенёв Илья Иванович. – Пермь: НИУ ВШЭ – Пермь, 2025. 69 с.

Работа посвящена реализации информационной системы для ускорения декодирования видеофайлов, представленных в формате AV1.

Во введении обосновывается актуальность данной работы, ставятся цель проекта и задачи. В первой главе приведены результаты анализа предметной области, в частности обзору стандарта AV1 Bitstream & Decoding Process Specification v1.0.0-errata и его реализации в существующем видеodeкодере Dav1d. Глава заканчивается анализом требований к данной системе. Во второй главе представлены результаты проектирования данной системы. Выполнено проектирование системы в общем и отдельных её компонентов в частности. Задачи проектирования решены на основе объектно-ориентированного подхода, при построении моделей использован язык UML. Третья глава описывает процесс реализации системы и оптимизации видеodeкодера Dav1d с указанием результатов оптимизации. В заключении описаны сделанные выводы и результаты работы.

Работа содержит 63 страницы основного текста, 6 таблиц, 44 рисунка. Библиографический список включает 27 наименований. Работа включает 1 приложение.

ОГЛАВЛЕНИЕ

Введение	5
Глава 1. Анализ спецификации av1 и технических средств повышения производительности.....	7
1.1. Обоснование актуальности и вывод первичных требований	7
1.2. Анализ текущего бизнес-процесса декодирования видео.....	8
1.3. Анализ существующих решений	10
1.4. Выделение функциональных требований.....	11
1.5. Определение вариантов использования системы.....	11
1.6. Выделение нефункциональных требований.....	14
1.7. Анализ стандарта AV1-v1.0.0errata.....	14
1.7.1. Сравнение с стандартом ITU-T H.264.....	14
1.7.2. Intra декодирование.....	18
1.7.3. Inter декодирование.....	22
1.7.4. Результаты анализа стандарта AV1	22
1.8. Анализ методов оптимизации кода	23
1.8.1. Правила Бентли	23
1.8.2. Оптимизация CPU-кеша.....	25
1.9. Анализ существующей инфраструктуры.....	26
1.10. Анализ технологического стека	29
1.10.1. Веб-сервер.....	29
1.10.2. Оркестрация.....	29
1.10.3. Базы данных.....	29
1.10.4. Брокер сообщений.....	30
1.10.5. Фронтенд.....	30
1.11. Результаты анализа объекта автоматизации	30
Глава 2. Проектирование наивной и оптимизированной систем	32
2.1. Проектирование наивной реализации системы	32
2.1.1. Проектирование базы данных.....	34
2.2. Проектирование оптимизированной реализации системы	34
2.2.1. Проектирование реляционной базы данных	38
2.2.2. Проектирование бэкенда	39
2.2.3. Проектирование фронтенда	44
2.2.4. Проектирование оптимизаций Dav1d	45
2.3. Выводы	48
Глава 3. Реализация оптимизаций и систем декодирования.....	50
3.1. Оптимизации DAV1D	50
3.1.1. CDEF, Loop filter (DF).....	50
3.1.2. Интерполяция	51

3.1.3. Оптимизация обратных трансформаций	55
3.1.4. Оптимизация реконструкции изображения.....	55
3.1.5. Выводы.....	56
3.2. Финальная система.....	57
3.2.1. Реализация наивной системы.....	57
3.2.2. Реализация оптимизированной системы	60
3.3. Тестирование	63
3.3.1. Общая задержка от начала отправки видео на сервер до полного декодирования	64
3.3.2. Минимальное время декодирования видео без учета времени передачи	64
3.3.3. Покадровая метрика потери качества видео.....	64
3.3.4. Покадровая метрика PSNR.....	65
3.3.5. Нагрузочное тестирование	66
3.4. Исходный код.....	66
Заключение.....	67
Глоссарий	68
Библиографический список.....	69

ВВЕДЕНИЕ

Индустрия кодирования видео является технически сложной областью, демонстрирующей впечатляющие результаты. Например, видеокодек H.264, чья первая версия стандарта была выпущена в 2003 году, демонстрирует снижение веса видео в 50-500 раз в сравнении с полным отсутствием сжатия видео [9].

Такая внушительная эффективность достигается благодаря тщательно продуманным стандартам с учетом архитектуры современных процессоров, математическим оптимизациям в области линейной алгебры, технологии кодирования motion vectors и способам предсказаний следующих кадров, и с каждым новым стандартом кодека эффективность лишь повышается.

Однако у такой высокой эффективности сжатия существует и обратная сторона, которая заключается в том, что с повышением уровня сжатия также повышается и необходимая процессорная нагрузка для декодирования файла.

Представленная выпускная квалификационная работа посвящена решению актуальной бизнес-задачи для компании ООО «КОМЭКСП» (<https://comexp.net/>). Основным продуктом данной компании – запатентованный индексатор видео TAPe, который позволяет осуществлять поиск видео по его фрагменту. Проблема заключается в том, что TAPe работает значительно быстрее любого видеodeкодера. Так, например, декодирование получасового фрагмента видеофильма занимает десять минут [25], а его индексирование происходит за десять секунд, в результате чего индексация видео занимает менее процента от общего времени работы системы.

Ввиду вышеперечисленных обстоятельств было принято решение разработать систему, чтобы максимально уменьшить простой индексатора относительно AV1 видео.

Данная работа выполняется в следующих условиях: процессор AMD EPYC 9554, SSD, однопоточный режим видеodeкодирования, выходное разрешение 64px пропорционально входному. Данный выбор обусловлен тем, что однопоточные оптимизации легче переносить на другие среды выполнения, они являются легковесными и могут запускаться на клиенте, а также потребляют меньше процессорных ресурсов чем многопоточная среда. NVIDIA CUDA не использована, так как её реализация будет выполнена в будущем и будет основываться на

оптимизациях в текущей работе. Сверхнизкое выходное разрешение обусловлено особенностями работы TAPe, о чем более подробно упомянуто в первой главе.

Объект исследования – процесс восстановления видеоданных из закодированного состояния.

Предмет исследования – оптимизированная система декодирования видео в формате AV1.

Цель исследования – разработать и применить оптимизации видеodeкодера AV1 для выходного разрешения 64px, спроектировать архитектуру системы декодирования с учетом максимальной производительности.

Для достижения указанной цели необходимо решить следующие задачи:

1. Выполнить анализ предметной области и на основе методов оптимизации ПО разработать требования к системе декодирования видео в формате AV1.
2. Выполнить проектирование архитектуры системы согласно требованиям для максимальной производительности.
3. Выполнить проектирование алгоритмов оптимизации видеodeкодирования AV1.
4. Выполнить программную реализацию системы, позволяющей декодировать AV1 видео, выполнить тестирование с целью оценки производительности.
5. Выполнить развертывание наивной и оптимизированной систем с использованием нескольких серверов, произвести тестирование пропускной способности.

Практическая значимость данной работы заключается в сокращении времени декодирования видео, что уменьшит время простоя индексатора TAPe, увеличит быстродействие системы и удешевит масштабирование продуктов, основанных на TAPe. На рынке не выявлены существующие решения, оптимизированные для декодирования видео в низком выходном формате

ГЛАВА 1. АНАЛИЗ СПЕЦИФИКАЦИИ AV1 И ТЕХНИЧЕСКИХ СРЕДСТВ ПОВЫШЕНИЯ ПРОИЗВОДИТЕЛЬНОСТИ

В ходе анализа должны быть решены следующие задачи:

1. Поиск потенциальных узких мест в стандарте Errata
2. Анализ методов оптимизации кода с целью их дальнейшего использования
3. Выбор оптимального технического стека для реализации системы
4. Определение функциональных и нефункциональных требований к системе
5. Формирование технического задания

1.1. Обоснование актуальности и вывод первичных требований

Компания «ООО КОМЭКСП», в интересах которой выполняется работа, оказывает услуги по поиску видео по видео с помощью технологии индексации TAPe. TAPe принимает на вход кадры видео, а на выходе выдает «индекс» видео, представляющий собой сжатый набор черт по которым видео можно отличать друг от друга и присваивать им индекс «похожести», называемый Т-индексом.

Сам процесс индексации происходит относительно быстро и ограничен по большей части скоростью чтения данных, однако декодирование видео с целью получения кадров из потока байтов происходит сильно медленнее. Например, индексация фильма «The Fate of the Furious» размером 3 гигабайта в разрешении 1920 * 1080 занимает 10 секунд, а его декодирование стандартным видеodeкодером libaom – 247 секунд.

Из вышесказанного следует, что имеет смысл оптимизировать именно процесс передачи видео на сервер и его последующее декодирование, так как он является «бутылочным горлышком» для всего процесса индексации.

Таким образом, к системе можно предъявить два важных первичных требования: возможность параллельно обрабатывать множество запросов, а также максимально сократить время, которое необходимо для того чтобы передать видео на сервер и декодировать его.

Система будет использовать видеodeкодер Dav1d для декодирования AV1 видео и он должен быть собран под архитектуру generic. Это значит, что в нем не будут использоваться процессор-специфичные оптимизации вроде SIMD, AVX, SSE2 для облегчения дальнейшего портирования. Также не будет использоваться Nvidia

CUDA ввиду того, что в данной работе первостепенной целью является алгоритмическая оптимизация, а не «железная».

При этом неважно качество работы системы на видео высокого разрешения, TARe одинаково хорошо работает для видео 128*128 и для видео 1920*1920, так что оптимизации будут проводиться с потерей качества.

1.2. Анализ текущего бизнес-процесса декодирования видео

Текущий бизнес-процесс декодирования видео представлен на рис. 1 и представляет собой последовательную авторизацию, загрузку видео на сервер внешнего API, затем его индексацию внутренним API. Внутреннее и внешнее API располагаются на одном сервере, что уменьшает сетевую задержку.

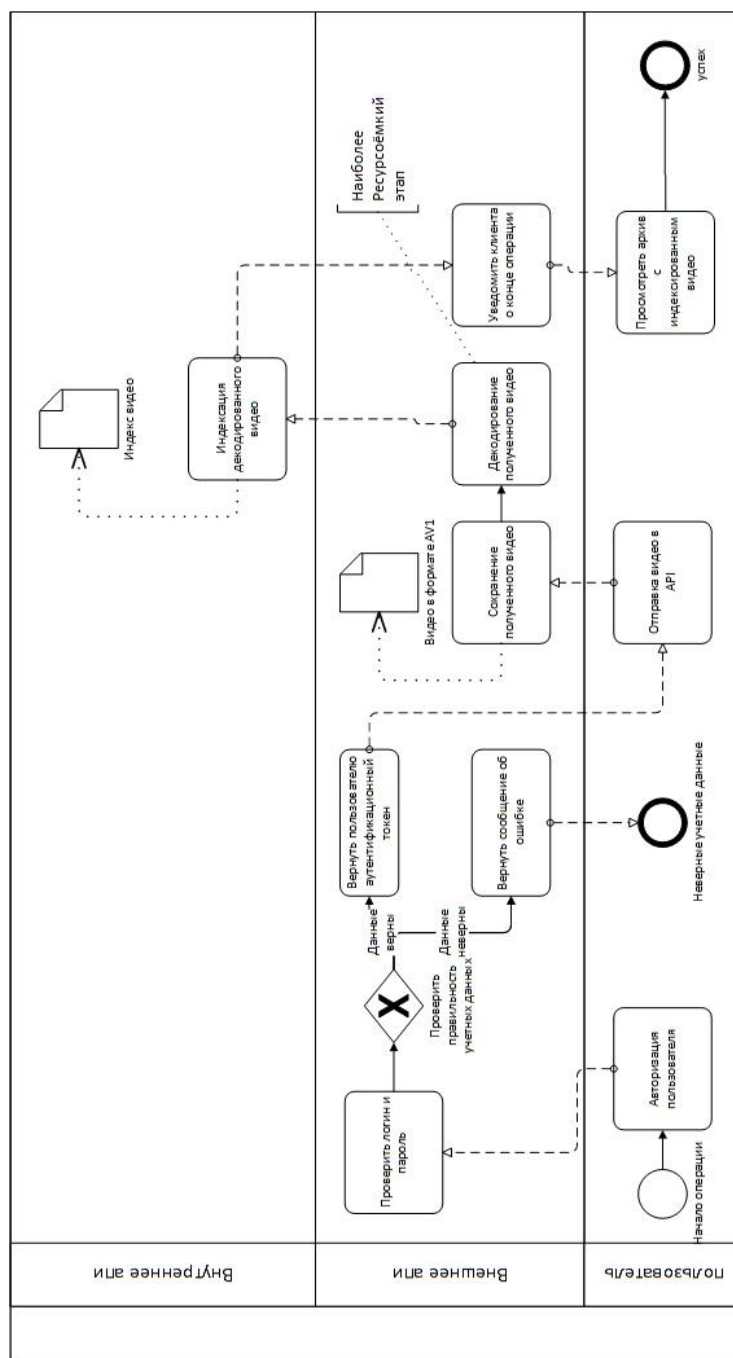


Рисунок 1. текущий бизнес-процесс

Как видно из схемы, внешнее API сохраняет видеофайл на сервер, а затем передаёт внутреннему API путь до этого файла, так как оба API находятся на одном сервере. После этого внутреннее API индексирует файл, сохраняет его и возвращает ответ.

Внешнее API представляет собой высокоуровневое API, предназначенное для взаимодействия с бизнес-сущностями, представленными таблицами в базе данных.

Внутреннее API закрыто для доступа извне и предназначено только для TAPe операций: поиск, индексация, сравнение и т.д.

Недостаток существующего решения в том, что оно крайне плохо поддаётся масштабированию: внутреннее API принимает на вход не файл, а его путь на сервере, оба API запущены в единственных экземплярах, для декодирования видео используется медленная референсная имплементация AV1 декодера libaom.

Наиболее ресурсоёмким местом является декодирование видео, что обозначено на схеме.

1.3. Анализ существующих решений

Система имеет в требованиях возможность полного декодирования видео в формате AV1 как можно быстрее, притом необязательно чтобы видео в высоком разрешении имело высокое выходное качество. Из этих двух требований следует что не может быть аналогов данной системы (по крайней мере, в открытом доступе) ввиду того, что пользователям в абсолютном большинстве случаев не нужно декодировать всё видео сразу, так как большинство видеodeкодеров поддерживают потоковое и покадровое декодирование. Ситуация осложняется тем, что к каждому из видеodeкодеров необходимо подбирать свой метод оптимизации, что достаточно трудоемко.

Ввиду вышеперечисленного в данном разделе представлен анализ существующих видеodeкодеров для видео в формате AV1 с их сравнительными характеристиками в таблице 1. Тестирование проведено с учётом genetic архитектуры, без использования SIMD на получасовом видео, с использованием процессора intel i5-12400f в однопоточном режиме. Код для воспроизведения доступен в репозитории <https://github.com/ressiwave/AV1-bench>.

Таблица 1 – сравнительный анализ видеodeкодеров [25]

Видеodeкодер	Скорость декодирования получасового видео, в секундах	Язык реализации
libaom	679.788	C
Dav1d	374.877	C
Rav1d	555.763	Rust, C

Ввиду вышеперечисленных факторов в качестве видеodeкодера был выбран Dav1d. Он предоставляет значительно большую скорость декодирования в отличие от референсной имплементации libaom и своего порта на языке Rust Rav1d.

1.4. Выделение функциональных требований

Для системы быстрого декодирования видео в формате AV1 были сформулированы функциональные требования:

- Система должна декодировать видео в формате AV1
- Система должна иметь возможность загрузки видео пользователем
- Система должна хранить метаданные о загруженных видеофайлах и статусах задач в реляционной базе данных.
- Система должна уведомлять пользователя о завершении задачи декодирования посредством протокола WebSocket.

1.5. Определение вариантов использования системы

Система будет иметь единственное предназначение, возможность провести видео через процесс «загрузка-декодирование» максимально быстро. Для этого процесса также необходима авторизация для идентификации пользователя. На рис. 2 представлена диаграмма прецедентов (вариантов использования) системы. На ней представлены существующие системы ComExp, для которых необходимо наличие декодированных файлов, а также рассматриваемой системы. В диаграмме представлены 4 сценария использования: зарегистрироваться, аутентифицироваться, отправить видео для декодирования и получить декодированное видео. На основе диаграммы для ключевого варианта использования «загрузка-декодирование» на рис. 3 приведена диаграмма последовательностей, детализирующая взаимодействие объектов.

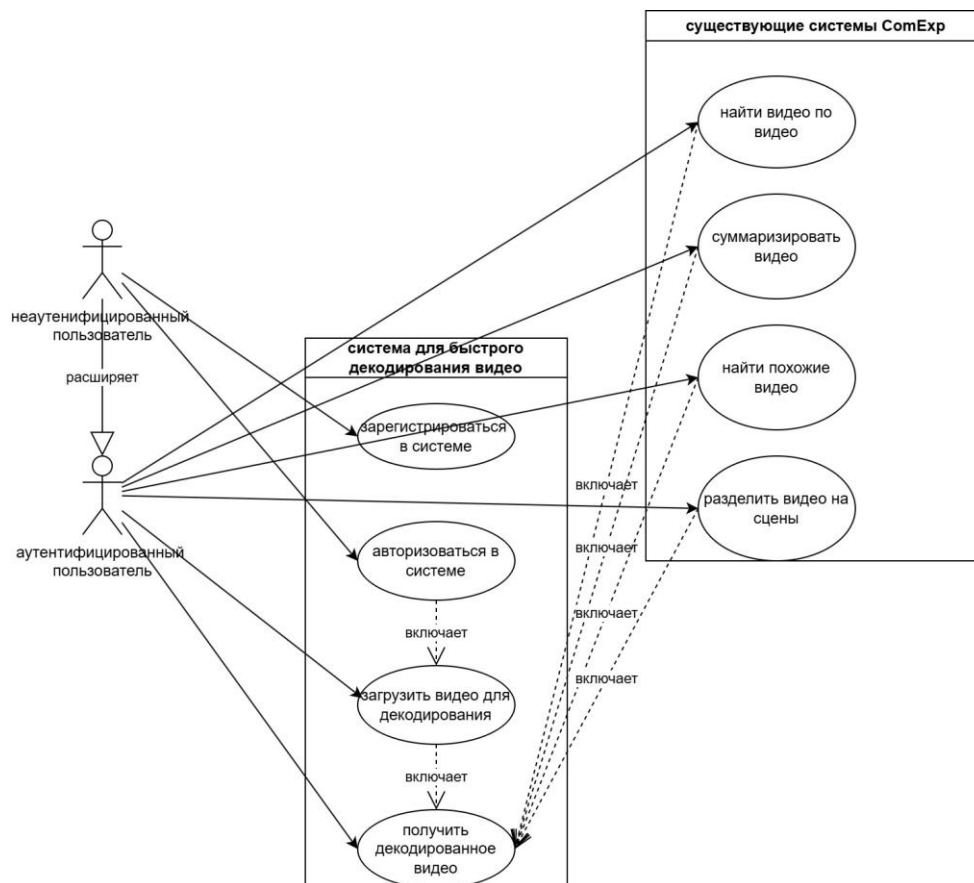


Рисунок 2. диаграмма прецедентов использования системы

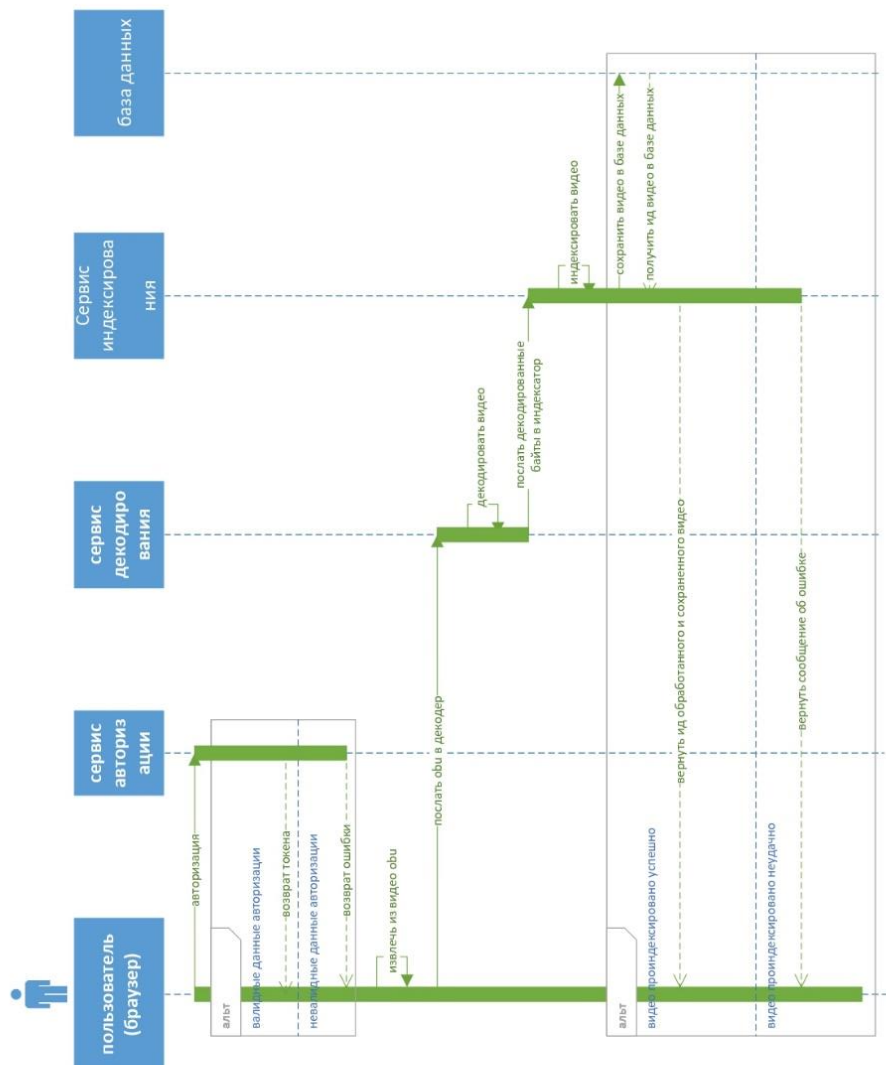


Рисунок 3. диаграмма последовательностей

Имеет необходимость пояснить термин «obu» в данной диаграмме: согласно AV1-v1.0.0errata obu расшифровывается как open bitstream unit и является неким блоком, из которых состоит видео. Тут, однако, имеется файл с расширением .obu, коим является видеодорожка, отделенная от аудио. Это одна из оптимизаций, которая будет объяснена и применена в главах 2 и 3.

1.6. Выделение нефункциональных требований

Для системы быстрого декодирования видео в формате AV1 были сформулированы нефункциональные требования:

- Нагрузка при обработке нескольких запросов должна линейно масштабироваться при превышении загруженности процессорных мощностей чтобы снизить задержки.
- Декодер AV1 должен быть собран под архитектуру generic и не использовать Nvidia CUDA.
- Оптимизированная система должна проходить путь загрузка-декодирование быстрее наивной реализации (см. главу 2) не менее чем на 20%.
- Оптимизированный видеodeкодер должен декодировать видео быстрее чем его неоптимизированная реализация не менее чем на 30%.

1.7. Анализ стандарта AV1-v1.0.0errata

В данной главе представлено описание стандарта Errata с целью выявления потенциальных узких мест для оптимизации. Также использовано сравнение с стандартом ITU-T H.264 для более наглядного объяснения, так как методы в последнем во многом служат опорой для Errata, однако более просты в понимании.

1.7.1. Сравнение с стандартом ITU-T H.264

Несмотря на то, что один из старейших стандартов кодека для H.264, появившийся в 2003 году, вышел раньше первого стандарта для AV1 на 15 лет (первая версия стандарта AV1 была опубликована в 2018 году), эти стандарты имеют много схожих черт, которые рассмотрены в данной главе.

На рис. 4 представлена базовая высокоуровневая схема декодирования по стандарту H.264. На нем можно увидеть, что декодирование может быть циклическим ввиду того, что некоторые кадры используют ранее декодированные кадры при реконструкции, а также что данные проходят через несколько развилок – способов предсказания [10].

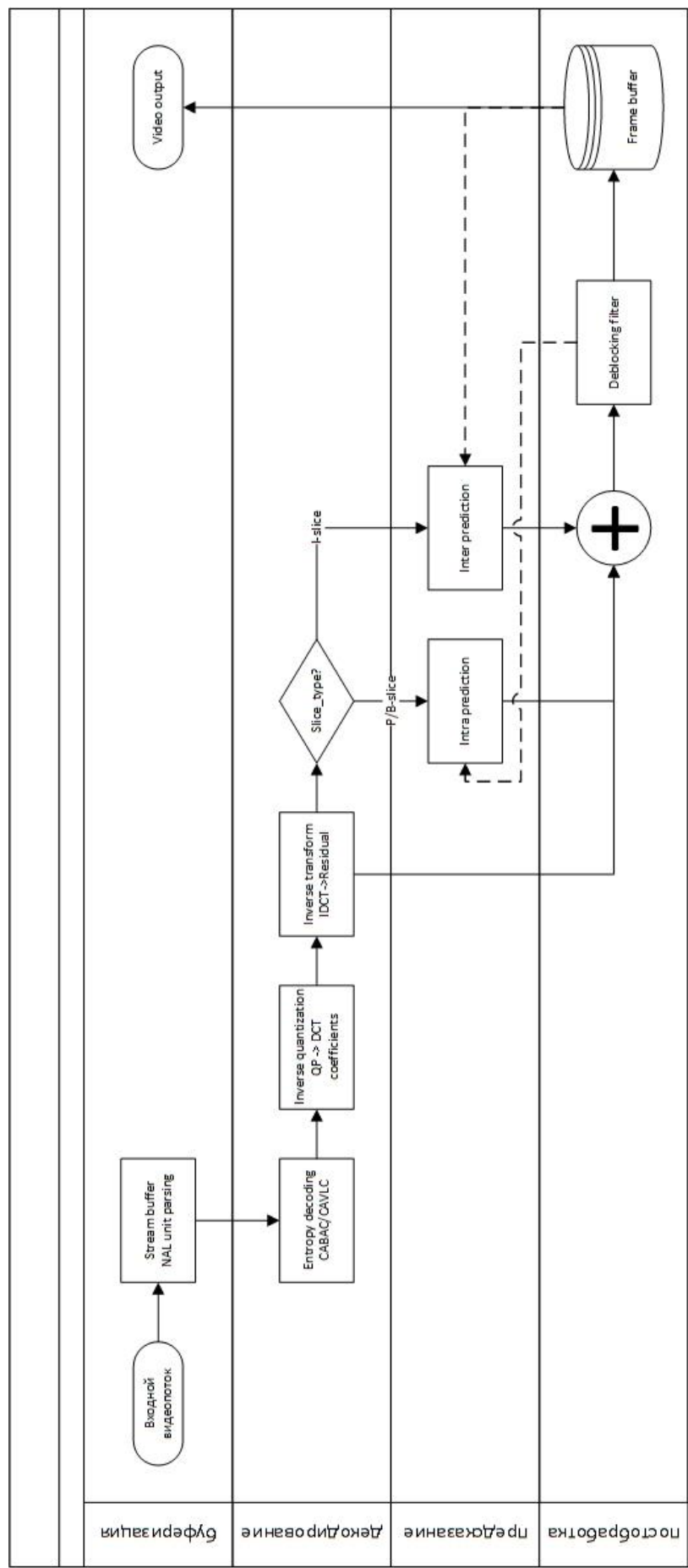


Рисунок 4.упрощённый бизнес-процесс декодирования видео H.264

Способы предсказания делятся на две важные категории: Intra предсказание и Inter предсказание. Первый метод – внутрикадровое предсказание, второе – межкадровое.

Также как в ITU-T H.264, в AV1-v1.0.0errata сохранилось intra предсказание, но стало более комплексным. В H.264 направление intra предсказания ограничено девятью углами от 216 градусов до 18 (см. рис. 5, 6) и действует в рамках макроблоков (см. раздел 1.7.2), в AV1 базовые концепты Intra предсказания сохраняются но действуют рекурсивно в рамках объединений макроблоков размерности 8*8, которые разбиваются внутри себя на блоки размерности 4*2. Эта оптимизация улучшает качество изображения и обеспечивает лучшую многопоточную производительность ввиду независимости 8*8 блоков но замедляет скорость декодирования.

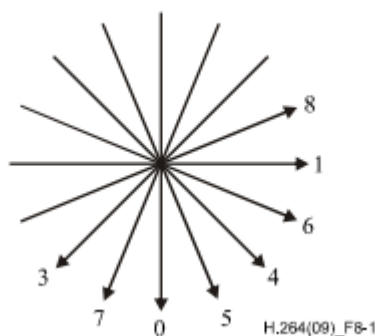


Рисунок 5. intra углы [18]

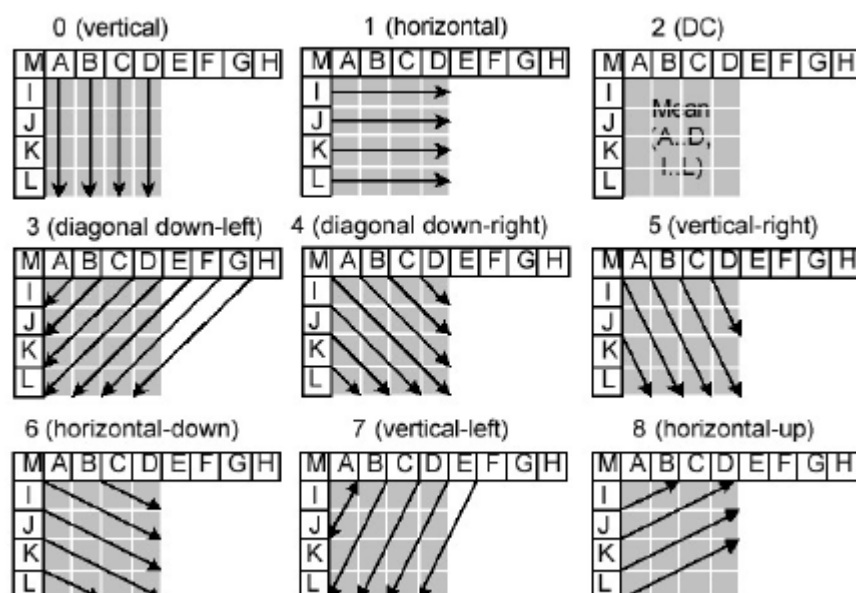


Рисунок 6. intra предсказание [19]

Касательно Inter предсказания – в H.264 этот способ работает так: существуют фреймы, являющиеся по сути кадрами видео. Фреймы делятся на несколько категорий, наиболее часто встречающимися являются *I*, *P*, *B* фреймы. *I*-фреймы независимо закодированы только intra методом, *P*-фреймы закодированы как разность motion-векторов (см. раздел 1.7.3) с предыдущим кадром, *B*-фреймы закодированы как разность motion-векторов с предыдущим и следующим кадром. AV1 не отделяет *I*-фреймы от *B*, *P*-фреймов, кодируя отдельный фрейм обоими способами, использует сильно более сложные методы предсказания и имеет новый вид фреймов – golden frame. Этот фрейм содержит в себе изображение высокого качества и часто используется для реконструкции. Также в AV1 вводится понятие суперфреймов, содержащих в себе несколько других фреймов, а один кадр может ссылаться не на 1-2 фрейма как в H.264, а иметь древовидную и комплексную структуру ссылок до 8 кадров, где каждому кадру соответствует временной слой (см. рис. 7, 8).

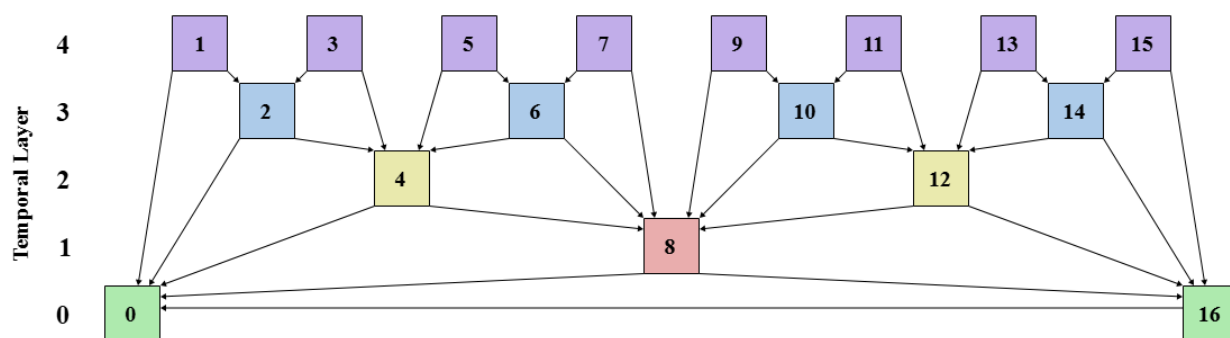


Рисунок 7. мультиреференсные кадры в intra-предсказании [11]

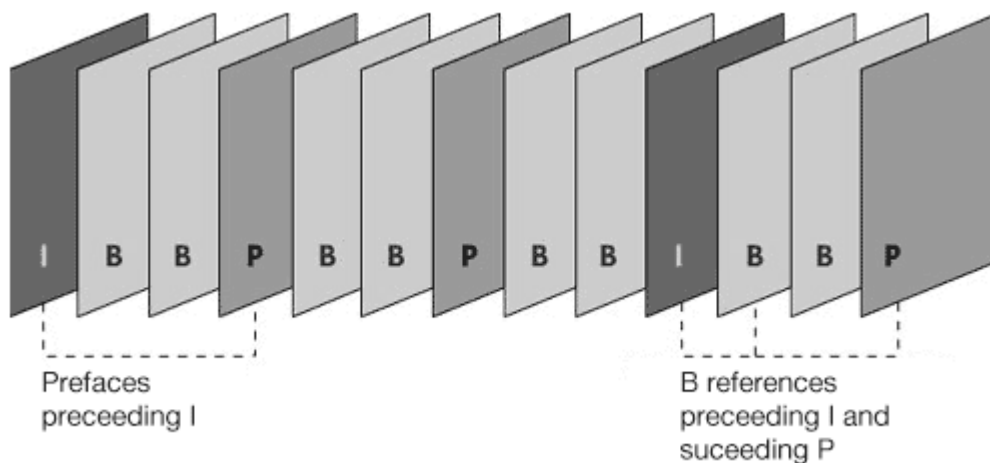


Рисунок 8. референсная система в H.264 [16]

1.7.2. Intra декодирование

Как уже было сказано, предсказания в кодеках делятся на 2 вида. В этой главе будут рассмотрены методы внутрикадрового предсказания.

1.7.2.1. Макроблоки

В стандарте ITU-T H.264 макроблоками называются массивы luma пикселей 16*16 и соответствующие им массивы Cb, Cr пикселей. Luma в данном случае означает яркость, Cb и Cr – цветовые компоненты синего и красного оттенков в формате YCbCr (Chroma-Luma). Макроблок – объект, которым оперирует inter или intra предсказание, а также множественные методы реконструирования и энтропийного декодирования.

В стандарте AV1-v1.0.0errata макроблоки заменяются суперблоками, однако общие принципы остаются теми же за исключением того что размерность суперблоков – 64*64.

1.7.2.2. Интерполяция

Интерполяция по своей сути в AV1-v1.0.0errata похожа на ту, что была в ITU-T H.264.

При расчете смещения пикселей с вычислением motion vectors может оказаться так, что смещение будет не на натуральное число пикселей, а на вещественное, и в этом случае результирующий пиксель рассчитывается как взвешенная сумма окружающих его пикселей. На рис. 9 представлен фрагмент из стандарта ITU-T H.264, поясняющий как именно вычислять интерполированный пиксель. Так, например, для пикселя в позиции *b* результирующее значение будет вычислено по формуле 1, а результирующее значение *h* – по формуле 2.

$$(E - 5 * F + 20 * G + 20 * H - 5 * I + J) / 32 \quad (1)$$

Где E, F, G, H, I, J – значения соответствующих пикселей, представленных на рис. 9

$$(A - 5 * C + 20 * G + 20 * M - 5 * R + T) / 32. \quad (2)$$

Где A, C, G, M, R, T – значения соответствующих пикселей, представленных на рис. 9

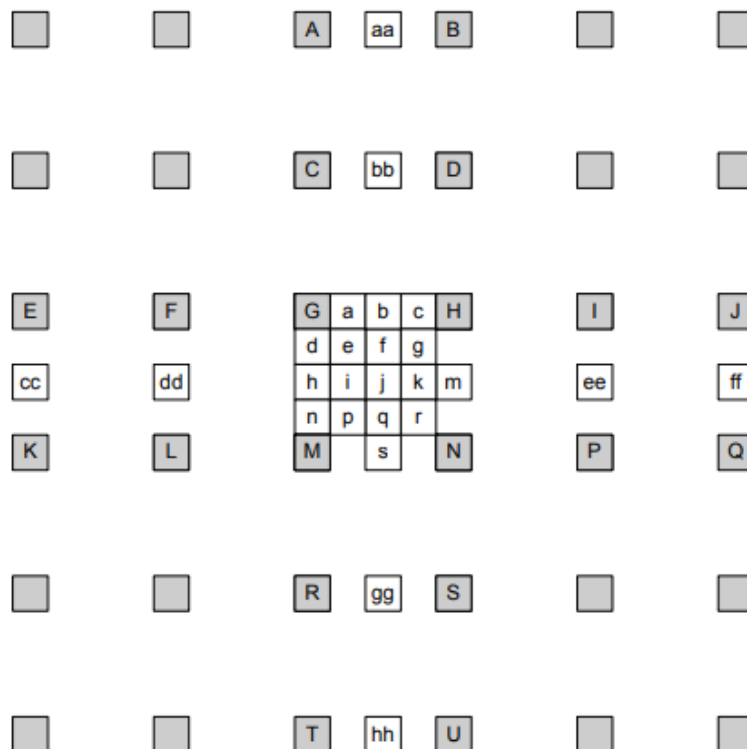


Рисунок 9. пояснение к методу вычисления интерполяции [17]

Данное вычисление крайне тяжелое и массово, даже при максимальных оптимизациях с сохранением логики будет потреблять огромное количество

процессорных тактов ввиду того, что даже для вычисления интерполяции всего лишь в одной плоскости понадобится минимум **22 такта** на x86 процессоре (не считая simd оптимизаций) [27]. Кроме 6-tap фильтра, описанного выше, существует обычная билинейная интерполяция и выбор первого или второго зависит от закодированной точности пикселя.

В стандарте AV1 интерполяция реализована похоже, однако коэффициенты (-1, 5, 10) динамические и закодированы в видео, притом всего вариантов коэффициентов 96 – по 16 на каждый метод интерполяции. В их числе обычная интерполяция, резкая, плавная и пр. Также в AV1 выше точность интерполяции и помимо 6-tap и билинейной интерполяции существуют 4-tap и 8-tap фильтры. Данная реализация на порядок «дороже» реализации H.264.

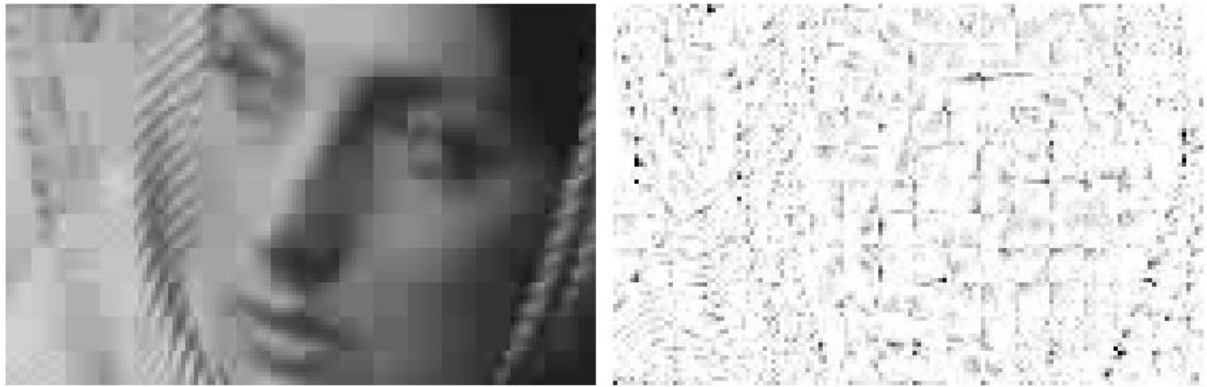
Данный тезис подтверждается статьёй [3], показывая связь интерполяции с остальными операциями. В статье говорится о том, что по сравнению с прямым предшественником AV1, VP9, её точность изменилась с 1/8 до 1/64. Это означает, что для расчёта одного пикселя с помощью 8-tap фильтра необходимо загрузить из памяти 8 пикселей и 8 коэффициентов, всего различных наборов коэффициентов – 96. Недостаток статьи в том, что она сосредоточена на кодировании, а не декодировании видео.

1.7.2.3. Энтропийное кодирование

В декодерах-наследниках MPEG (к коим относятся рассматриваемые AV1 и H.264) также присутствуют методы сжатия путем энтропийного кодирования вроде DCT, ADST, FLIPADST, IDTX. Не вдаваясь в детали, это методы кодирования, при котором видеodeкодер берет из потока байтов 2 коэффициента которые являются натуральными числами от 0 до 7, а затем переводит их в частоты через функцию косинуса (для discrete cosine transform). Так, например, из коэффициентов (0,0) будет восстановлена однородно-серая текстура, из (1,0) – горизонтальный градиент, из (7,7) – мелкую сетку. Это позволяет сильно сэкономить занимаемое место и время декодирования, так как методы декодирования сильно оптимизированы даже в виде нативного кода на си.

1.7.2.4. Технология CDEF

В AV1 существует такая технология как constrained directional enhancement filter (CDEF). CDEF представляет собой фильтр пост-обработки, который наряду с deblocking filter (DF) улучшает качество изображения. Выше были рассмотрены методы энтропийного кодирования, однако у них есть неприятное побочное действие: накопление ошибок на краях блоков. Восстановленная таким образом картинка похожа на мозаику из квадратов, как на рис. 10.



Closeup of reconstructed image

Normalized error distribution within each block

Рисунок 10. ошибки инверсной дискретной реконструкции

Для решения этой проблемы существуют 3 фильтра, применяемых в каждом кадре по порядку: loop filter, CDEF. Конкретно CDEF отвечает за то, чтобы убрать шум рядом с границами. Для этого вычисляется, насколько текущий блок пикселей попадает в каждый из 8 фильтров направлений (45° , 22.5° , 0° , -22.5° , -45° , -67.5° , -90° , -112.5°) и дальше по границам идет фильтр, похожий на интерполяционный. В статье [3] предоставлена формула (7) фильтра по которой возможно оценить потенциальную трудоёмкость операции.

$$y(i, j) = R(x(i, j) + g(\sum_{m, n \in N} w_{m, n} f(x(m, n) - x(i, j), S, D))) \quad (7)$$

Где N содержит пиксели соседние с $x(i, j)$ с не-нулевыми весами $w_{m, n}$,

$f()$ и $g()$ – нелинейные функции,

$R(x)$ – округление x к ближайшему целому.

1.7.2.5. Технология Deblocking filter

Наряду с CDEF, DF устраняет блочность изображения, также за счет сложной интерполяции. Например, в работе [20] показана оптимизация адаптивного фильтра деблокинга, который работает на границах блоков 4×4 . Его вычислительная

сложность, несмотря на относительную простоту операций (сдвиги, сложения), остается высокой из-за необходимости обработки каждого пикселя на границе. В отличие от H.264, кодек AV1 использует более сложный и многоступенчатый подход к фильтрации, описанный подробно в главе VII статьи [21], что делает прямую репликацию методов невозможной. Однако, ключевая идея оптимизации — использование контекстной информации из битового потока для адаптации вычислительной нагрузки — остается актуальной для данной работы. Тем не менее, работа несет риск плохой воспроизводимости ввиду того, что используемые методы были применены на референсном декодере JM9.5, отличающемся тем, что в нём реализован необходимый минимум из стандарта. Это значит, что в нём изначально был большой потенциал оптимизаций и применение данной методики к AV1 может не дать существенного результата. Важно отметить, что DF и CDEF – дорогие операции для улучшения качества изображения в высоком разрешении.

1.7.3. Inter декодирование

1.7.3.1. Технология Motion Vectors

Motion vectors (MV) – один из важнейших способов сжатия в видеодекодерах, позволяющий не хранить все кадры. В видео почти всегда присутствует движение, и ввиду этого вместо того чтобы кодировать каждый кадр как изображение, возможно закодировать движение пикселей или областей пикселей. Например, есть видео в котором мяч летит в ворота, а камера неподвижна. Вместо того, чтобы отдельно кодировать каждый кадр кодируется начальный кадр, а дальше – только дельты пикселей мяча для каждого кадра. Опорные кадры (также называющиеся reference frames или IDR frames) – кадры, которые невыгодно кодировать с помощью motion vectors, например смена сцены или положения камеры.

1.7.3.2. Сущности Frames

Подробная информация о фреймах представлена в разделе 1.7, можно лишь добавить что в AV1 существуют hidden frames – опорные фреймы, которые не добавляются в итоговый результат.

1.7.4. Результаты анализа стандарта AV1

После проведенного анализа возможно выделить следующие средства, на которые стоит обратить внимание при оптимизации, представленные в таблице 2.

Таблица 2 – сравнительная характеристика ключевых процессов AVI

таблица	Почему должно быть оптимизировано
CDEF	Тяжеловесная формула (7), результат не виден в низком разрешении
DF	Применяется к каждому блоку, результат не виден в низком разрешении
Интерполяция	Применяется к каждому пикселю, занимает минимум 22 такта на пиксель
Энтропийное кодирование	Имеет смысл оптимизация для низкого разрешения с квантизацией коэффициентов

Данные операции занимают много ресурсов и должны быть оптимизированы в первую очередь. В перспективе это даст хорошее увеличение производительности. В главе «реализация» приведено профилирование данных частей стандарта.

1.8. Анализ методов оптимизации кода

В данной главе будет рассказано об основных методах оптимизации с опорой на «правила оптимизации», предложенные профессором Массачусетского Технологического Института Джоном Луисом Бентли. В главе «Реализация» они будут применены для сокращения времени декодирования видео.

1.8.1. Правила Бентли

Правила Бентли представляют собой подобие чек-листа из 20 пунктов, разделённых на 4 раздела: структуры данных, логика, циклы и функции. Некоторые пункты потеряли актуальность в связи с тем, что GNU GCC компилятор применяет их при включенных флагах -O2, -O3, а некоторые неприменимы к данной задаче. Рассмотрим оставшиеся.

1.8.1.1. Инициализация во время компиляции

Данный метод представляет собой практику инициализации значений во время компиляции. Например, есть функция, вычисляющая синус, умноженный на косинус, а точность выходного значения не сильно важна. В этом случае возможно инициализировать массив вида

```
uint32_t cossin[10]={0.0, 0.476, 0.294, -0.294, -0.476, -0.0, 0.476, 0.294, -0.294, -0.476}
```

(3)

затем вычислять приблизительное значение как

```
result = cossin[i]
```

(4)

где $i - (\pi * 2 / 10) * (\text{требуемый угол})$

Вычисляемые значения будут с погрешностью входного значения $< \pi * 1/5$. Тем не менее, данная методика может понижать производительность. В статье [22] рассматривается энтропийный декодер CAVLC кодека H.264, который, как и AV1, VP9, HEVC(H.265) является наследником MPEG и берет некоторые его концепции, а следовательно некоторые идеи по оптимизации H.264 могут быть полезны и для AV1. В частности, в статье рассматриваются такие методы оптимизации как аппаратное ускорение и отказ от lookup tables, пример которой представлен в данном разделе. Второй раздел более полезен для данной работы. В нём рассказывается о том, что отказ от множества lookup tables в пользу алгебраического выражения быстрее, так как это избавляет от необходимости в множественных вычислениях и кеш-промахх при обращении к данным.

1.8.1.2. Плотность

Как будет рассмотрено в разделе 1.9.2, крайне важно оптимизировать код для процессорного кеша, ввиду этого необходимо соблюдать плотность данных. Например, представим что в примере выше массив был инициализирован с точностью входного значения менее $\pi * 1/500$, тогда в массиве будет 1000 значений. Если код будет запрашивать элементы в соответствии с нормальным распределением и если учесть что линия процессорного кеша вмещает 8 его элементов, то процент промахов может достигнуть 87.5%. Если же элементов 10, то при прочих равных процент промахов не будет выше 20%.

1.8.1.3. Объединение тестов

Общеизвестно что процессор имеет механизм branching. Это значит, что процессор может предсказать результат логического ветвления в коде и в соответствии с ним «положить» в конвейер инструкций соответствующий участок кода. Чем меньше процент верно угаданных развилок, тем ниже производительность, и соответственно выгоднее всего сократить количество возможных развилок объединив несколько условий в одно.

1.8.1.4. Упорядочивание проверок

В языках программирования логические операторы, как правило, «ленивые», следовательно необходимо упорядочивать проверки в условиях так, чтобы конечный результат был известен как можно раньше.

1.8.1.5. Логические преобразования

Данный метод не входит в перечень оптимизаций Бенгли, однако его можно отнести к подвиду объединения тестов. Суть логических преобразований в том, чтобы заменить обычные логические вычисления быстрыми битовыми вычислениями.

Например, условие

`if (x<4096 && x>=256)` (5)

можно заменить условием

`if( (x|~0xFF)&&!(x&~0xFFF)) ).` (6)

Данный подход рассмотрен в статье [23]. В ней рассматриваются метод деления при помощи «магической константы» и его оптимизированный вариант, работающий на 30% быстрее. Деление редко встречается в декодировании видео и больше присутствует в кодировании, однако этот метод, несмотря на его сложность, может помочь ускорить интерполяцию, так как она производится путем деления суммы произведений коэффициентов фильтра и значений пикселя.

1.8.2. Оптимизация CPU-кеша

CPU-кеш – один из главных методов оптимизации кода. Его правильное использование может уменьшить скорость выполнения кода на величину до 99% в синтетических задачах. На рис. 11 представлена относительная скорость доступа к разным типам памяти, и из него понятно, что доступ к ячейке оперативной памяти DDR4 в 120 раз более долгий чем доступ к ячейке L1 процессорного кеша. Следовательно, максимальная выгода в синтетической задаче от перемещения данных из dram в L1 кеш более 99%.

Разумеется, это верно только для нескольких килобайт данных (48кб для intel i5-12400f), так что необходимо особо тщательно подходить к порядку запроса данных и количеству запрошенных данных.

Event	Latency	Scaled
1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access (DRAM, from CPU)	120 ns	6 min
Solid-state disk I/O (flash memory)	50–150 μ s	2–6 days
Rotational disk I/O	1–10 ms	1–12 months

Рисунок 11. memory latency table [24]

1.9. Анализ существующей инфраструктуры

В силу действующего контракта между «ООО КОМЭКСП» и исполнителем данной работы будет рассмотрена только структура существующей базы данных.

На рис. 12 8 представлена ER-диаграмма существующей базы данных. Из нее видно, что каждый архив связан с пользователем, а каждое видео — с архивом. На данный момент система позволяет объединять несколько видео в одну бизнес-сущность — архив.

Подробное описание предназначений полей предоставлено в таблице 3.

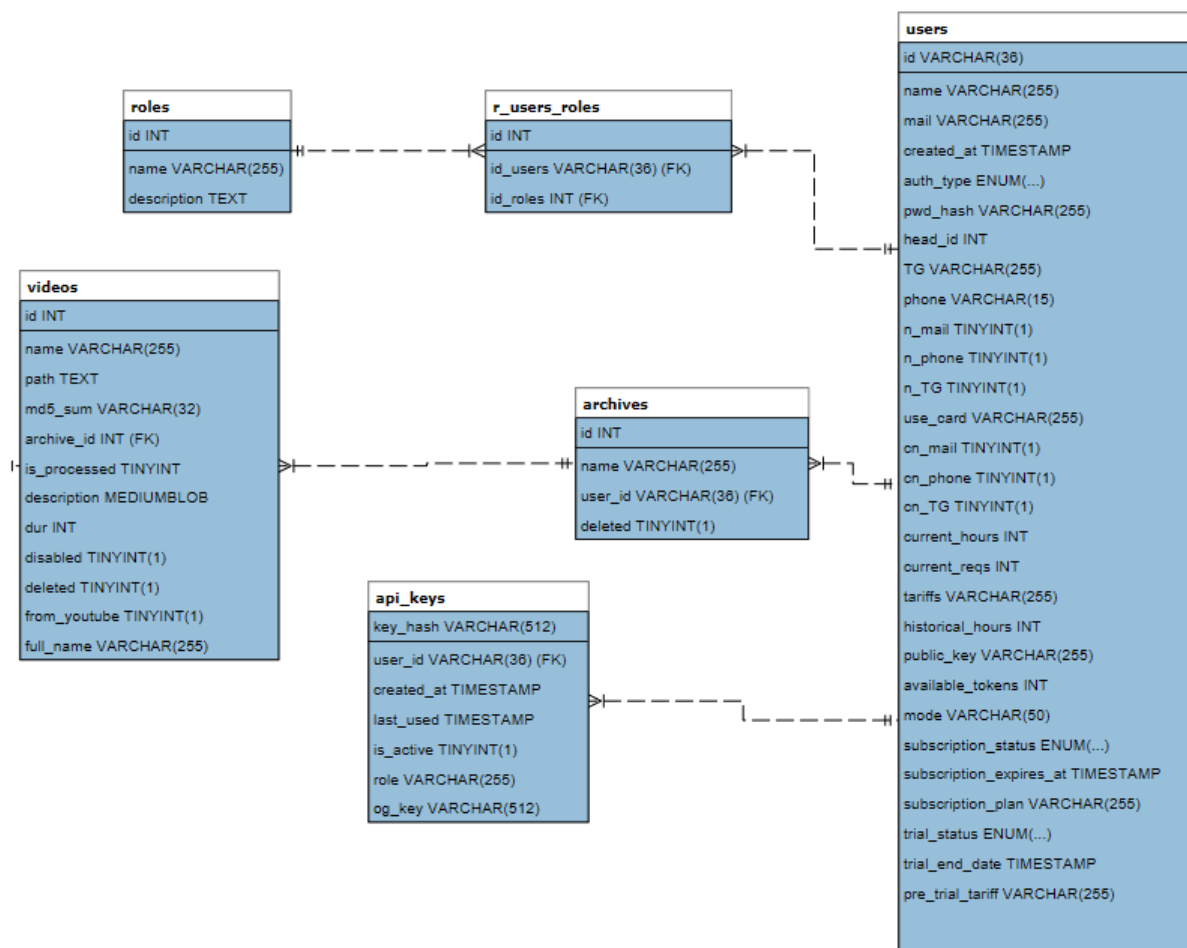


Рисунок 12. ERD базы данных

Таблица 3 – пояснение к ERD

таблица	Поле	Тип данных	Для чего нужно
Любая таблица	Id	int	Идентификатор записи
users	Name	String	Имя пользователя
	Mail	String	Почта пользователя
	Created_at	Timestamp	Когда был создан аккаунт
	Auth_type	String	Тип аутентификации
	Pwd hash	String	Хэш пароля
	TG	string	Идентификатор в Телеграм
	Phone	String	Номер телефона
	Current_hours	Int	Количество часов, в течение которых пользователь может использовать ТАРе
	Tariffs	String	Тариф пользователя

таблица	Поле	Тип данных	Для чего нужно
	Public_key	String	Публичный ключ пользователя
	Available_tokens	Int	Доступное количество токенов для поиска
	Subscription_status	string	Статус подписки
	Subscription_expires_at	Timestamp	Когда кончается подписка
	Subscription_plan	String	План подписки
	Trial_status	String	Статус пробного периода
	Trial_end_date	Timestamp	Конец пробного периода
	Pre_trial_tariff	String	Какой тариф был перед пробным периодом
archives	Name	String	Название архива
	User_id	Int	Идентификатор владельца архива
	Deleted	Bool	Удален ли архив
Api_keys	Key_hash	String	Хэш апи ключа
	User_id	Int	Идентификатор пользователя ключа
	Created_at	Timestamp	Когда был создан ключ
	Last_used	Timestamp	Последнее использование ключа
	Is_active	Bool	Активен ли ключ
	Role	String	Роль ключа (администратор, владелец, пользователь)
videos	Name	String	Название видео
	Path	String	Путь, по которому расположено видео
	Md5_sum	string	Md5 сумма видеофайла
	Archive_id	Int	Ид архива, которому принадлежит видео
	Is_processed	Bool	Проиндексировано ли видео
	Description	String	Описание видео
	Dur	Int	Длительность видео в секундах

таблица	Поле	Тип данных	Для чего нужно
	disabled	Bool	Отключено ли видео
	Deleted	Bool	Удалено ли видео
	From_youtube	Bool	Скачано ли видео с Youtube
	Full_name	String	Полное название видео

1.10. Анализ технологического стека

Для того, чтобы построить быструю и правильно работающую систему, которую легко поддерживать необходимо выбрать подходящие технологии. В данной главе будут перечислены основные технологии необходимые для реализации проекта.

1.10.1.Веб-сервер

В качестве веб-сервера был выбран фреймворк **FastApi**. Несмотря на то, что он написан на Python и в связи с этим имеет проблемы с Requests Per Second, он является одним из самых быстрых веб-фреймворков для Python [15] ввиду минимального оверхеда, в отличие например от Django с большим количеством слоев абстракций и синхронной блокирующей архитектурой.

1.10.2.Оркестрация

Учитывая требование о масштабируемости системы, она будет расположена на нескольких серверах для того чтобы обеспечить линейную загруженность пропорционально запросам. В связи с этим необходимо оркестрировать серверы, и для этого лучше всего подходит **Docker swarm** ввиду легкости развертывания. В дальнейшем может быть произведена миграция на Kubernetes.

1.10.3.Базы данных

Немаловажным фактором в задержках является также база данных. Для хранения «горячих данных» вроде токенов (Api_keys) пользователя будет использован **Redis**, так как он, храня информацию в оперативной памяти, уменьшает задержки памяти с уровня ssd/hdd до уровня оперативной памяти, что, исходя из таблицы на рис. 11, может снизить задержку доступа к данным на 1-2 порядка.

Для некритичных данных (Videos, Users, Archives) используем **PostgreSQL** ввиду её субъективного удобства и хорошего потенциала к масштабируемости, а для файлов используем S3 хранилище **Ceph**.

Следует пояснить, почему выбрано именно S3 хранилище для файлов вместо обычной файловой системы. Даже если бы в работе была не распределенная архитектура с одним сервером-монолитом, это было бы ненадежно и нестабильно. Файловая система ext4, являющаяся системой по умолчанию, не справляется с большим количеством (больше миллиона) директорий и файлов, что было выявлено в «ООО КОМЭКСП» экспериментально. К тому же, обращение к файлам по их путям в коде достаточно неудобно. В свою очередь, Ceph имеет в себе оптимизации для хранения множества мелких файлов.

1.10.4. Брокер сообщений

Учитывая распределённую архитектуру системы понадобится также брокер сообщений. Большинство брокеров используют под капотом быстрый протокол NATS, являющийся надстройкой над QUIC, так что будет выбран брокер с минимальным оверхедом – **NATS Jetstream**. Он характеризуется минимальными задержками, выдавая 1-5мс с гарантией доставки против 10-50мс у kafka [7].

1.10.5. Фронтенд

В качестве фреймворка для фронтенда выберем **React**, так как он фактически на данный момент является монополистом рынка среди других фреймворков [8] и предлагает интеграцию с WASM. Для ускорения передачи видео будет использоваться **ffmpeg**, скомпилированный в **WebAssembly** для отделения аудио от видео.

1.11. Результаты анализа объекта автоматизации

В ходе работы была проанализирована предметная область, представляющая собой способы кодирования видео, основные методы оптимизации и краткий разбор технологий, которые будут использоваться. Также были перечислены потенциальные узкие места в стандарте видеокодека AV1 и основные методы оптимизации согласно профессору Массачусетского Технологического Института Джону Луису Бентли.

Данная информация в будущем поможет при проектировании и реализации системы для декодирования видео.

В результате анализа предметной области было составлено техническое задание, представленное в приложении А.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ НАИВНОЙ И ОПТИМИЗИРОВАННОЙ СИСТЕМ

В данной главе будет представлено проектирование системы для декодирования видео в формате AV1. Несмотря на то, что система предназначена только для одной функции, необходимо будет реализовать вокруг неё полноценную инфраструктуру, включающую в себя базу данных, брокер сообщений, фронтенд и бэкенд.

Для последующего анализа эффективности оптимизация также будет спроектирована наивная реализация системы, без каких-либо оптимизаций. Анализ производительности будет производиться по следующим параметрам:

- Общая задержка от начала отправки видео на сервер до полного декодирования видео, в секундах.
- Минимальное время декодирования видео без учета времени передачи видео по результатам пяти тестовых запусков.
- Покадровая метрика потери качества видео, вычисляемая как разница Luma значений каждого пикселя кадра видео.
- Покадровая метрика PSNR.
- Время ответа при нагрузочном тестировании системы.

Целевыми метриками считаются ускорение метрик 1 и 2 на 30-50% и 20-35% соответственно в сравнении с наивной реализацией, при этом PSNR должно сохраняться выше 15дб. Метрики 3 и 5 необходимы для характеристики системы в целом, но не критичны.

2.1. Проектирование наивной реализации системы

Наивная реализация системы будет спроектирована максимально просто. Она будет представлять собой простое FastApi-приложение, также включающее в себя frontend как jinja templates. На рис. 13 представлена диаграмма контейнеров наивной системы. Как можно увидеть, центральным является контейнер с FastApi приложением, включающий в себя весь функционал. Рассмотрим данный контейнер детальнее на рис. 14.



Рисунок 13. naive system container diagram

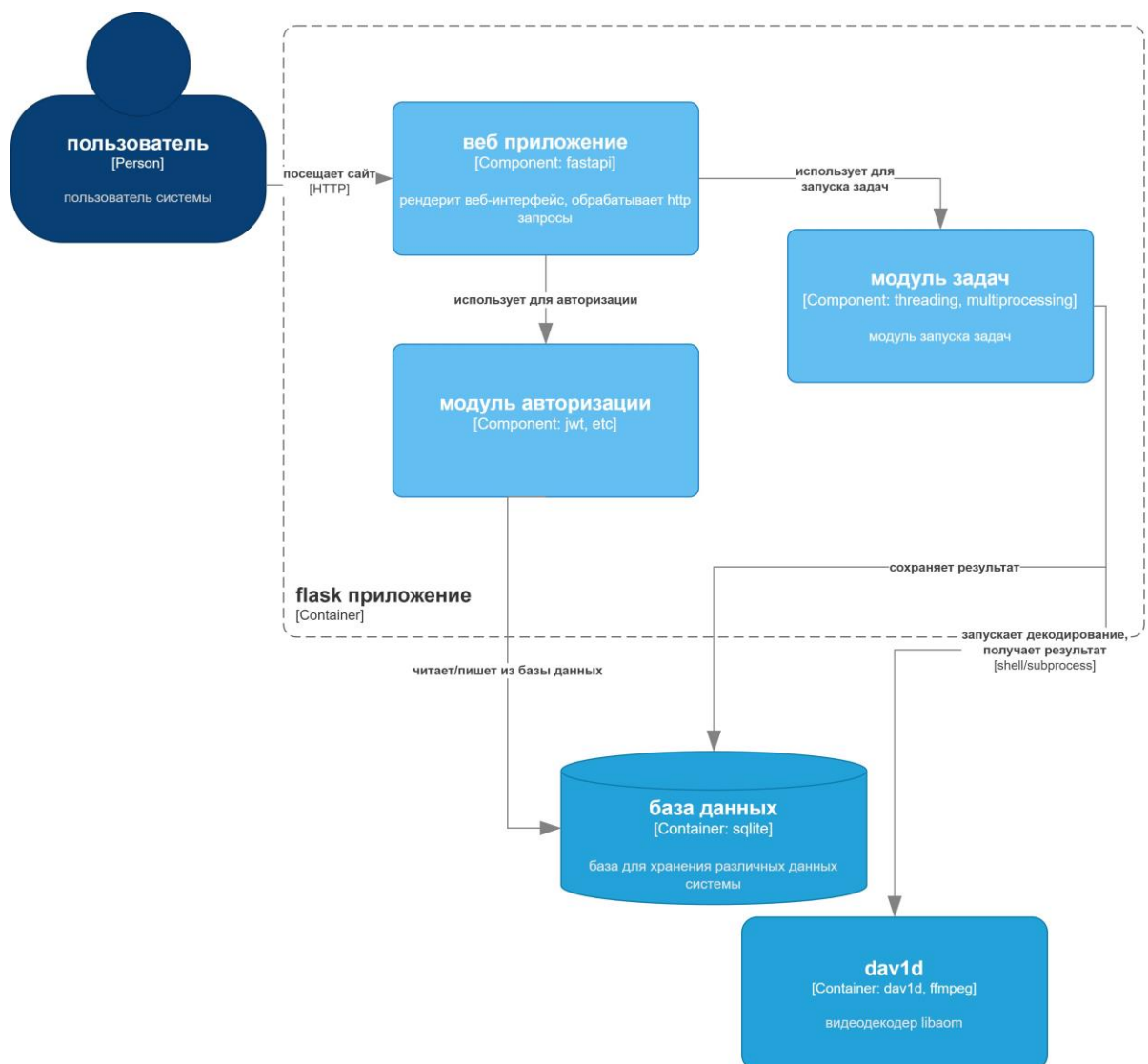


Рисунок 14. диаграмма контейнера FastApi приложения

На данной диаграмме показано, как контейнер будет выглядеть внутри. Он будет использовать обыкновенную jwt-систему для авторизации, токены при этом

будут храниться в sqlite. Планирование задач декодирования будет выполнено с помощью встроенных в python3 средств, таких как threading и multiprocessing.

2.1.1. Проектирование базы данных

Учитывая, что наивная реализация необходима только для сравнения с оптимизированной реализацией, оставим только необходимые поля таблиц. В результате получим следующую ER диаграмму, представленную на рис. 15. В данной схеме есть 4 таблицы.

Таблица Users предназначена для хранения информации о пользователях, идентификация производится по номеру телефона и паролю, притом пароль не хранится в открытом виде.

Таблица archives является бизнес-сущностью, объединяющей несколько видео. У архива есть название и идентификатор, притом архив может принадлежать одному пользователю.

Таблица Videos хранит метаданные о видео, включая его путь в файловой системе на сервере, md5 сумму и состояние, в котором оно находится.

Таблица Api_keys хранит в себе токены для авторизации пользователя.

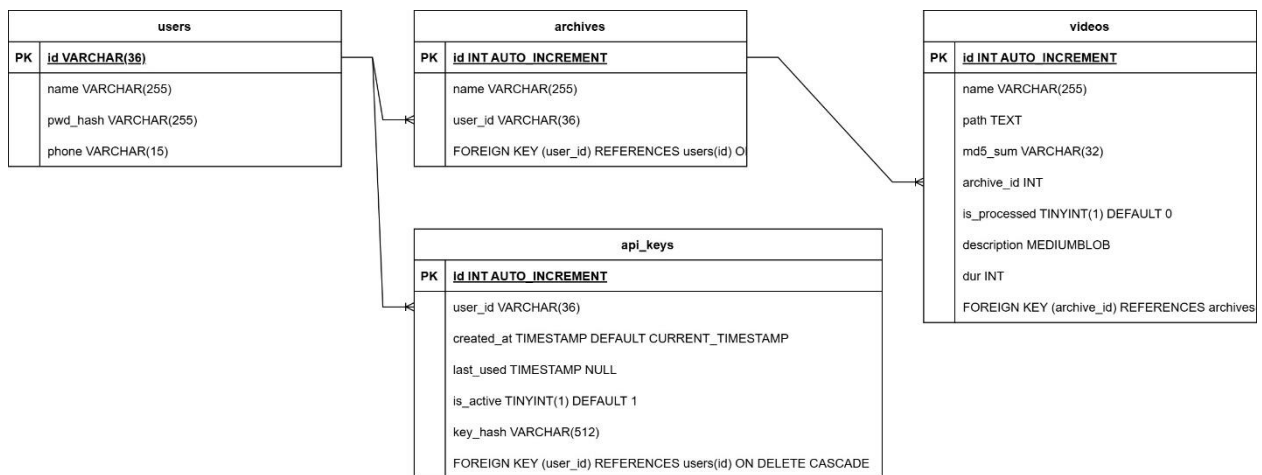


Рисунок 15. ER диаграмма базы данных наивной реализации

2.2. Проектирование оптимизированной реализации системы

Наивная архитектура, рассмотренная выше, представляет собой неоптимизированную монолитную систему, которую крайне сложно масштабировать. В данной главе будет рассмотрена реализация максимально быстрой и надежной версии системы.

Прежде всего необходимо будет сократить IO издержки, связанные с передачей файлов по сети. Учитывая, что система необходима для поиска видео по видео, объем файлов в некоторых случаях может достигать двух гигабайт. Для декодирования видео не нужна звуковая дорожка, присутствующая почти в каждом видеофайле, что поможет в оптимизации передачи видео. В связи с этим понадобится на стороне клиента вырезать из видеофайла аудиодорожку и отправлять в бэкенд получившийся файл. Также будет использован React как фреймворк для фронтенда ввиду его популярности и что более важно, масштабируемости.

Как было показано на рис. 11, длительность обращения к оперативной памяти может быть сильно быстрее обращения к памяти на жестком диске, это означает, что если видео будет сохранено в файловую систему и будет декодировано оттуда, то это будет медленнее чем декодирование из оперативной памяти. Для минимизации данной задержки используем принцип *memory buffer*, то есть декодирование из оперативной памяти напрямую. Данный подход работает не всегда, особенно при высоких нагрузках, и в этом случае всё-таки понадобится сохранить файл. Учитывая распределённую архитектуру, используем S3 хранилище. Оно необходимо, учитывая, что нужно будет распределить хранилище по нескольким серверам, что решит проблему ограниченности памяти.

Backend же тоже будет распределён по нескольким серверам. Для того, чтобы обеспечить согласованность серверов и доставку сообщений будет использован брокер сообщений NATS Jetstream. На таблице 4 продемонстрировано сравнение популярных брокеров сообщений. Стоит отметить, что очередь необходима также для асинхронных коммуникаций, а потому необходима персистентность сообщений. Стоит отметить, что NATS Jetstream – не идеальный брокер, который, в отличие от Apache Kafka, появился сравнительно недавно и поэтому у некоторых пользователей Jetstream появляются проблемы со стабильностью. Тем не менее, это один из самых быстрых брокеров, что имеет первостепенное значение для решаемой задачи.

Таблица 4 – сравнительный анализ брокеров сообщений

Брокер сообщений	Латентность	Сравнительная пропускная способность	Персистентность
Core NATS	<1мс [13]	Высокая	Нет

NATS JetStream	1-5мс [12]	Выше среднего	Да
Apache Kafka	10-50мс [12]	Высокая	Да
RabbitMQ	5-20мс [14]	Средняя	Да

Система представлена в виде диаграммы C4 на рис. 16.

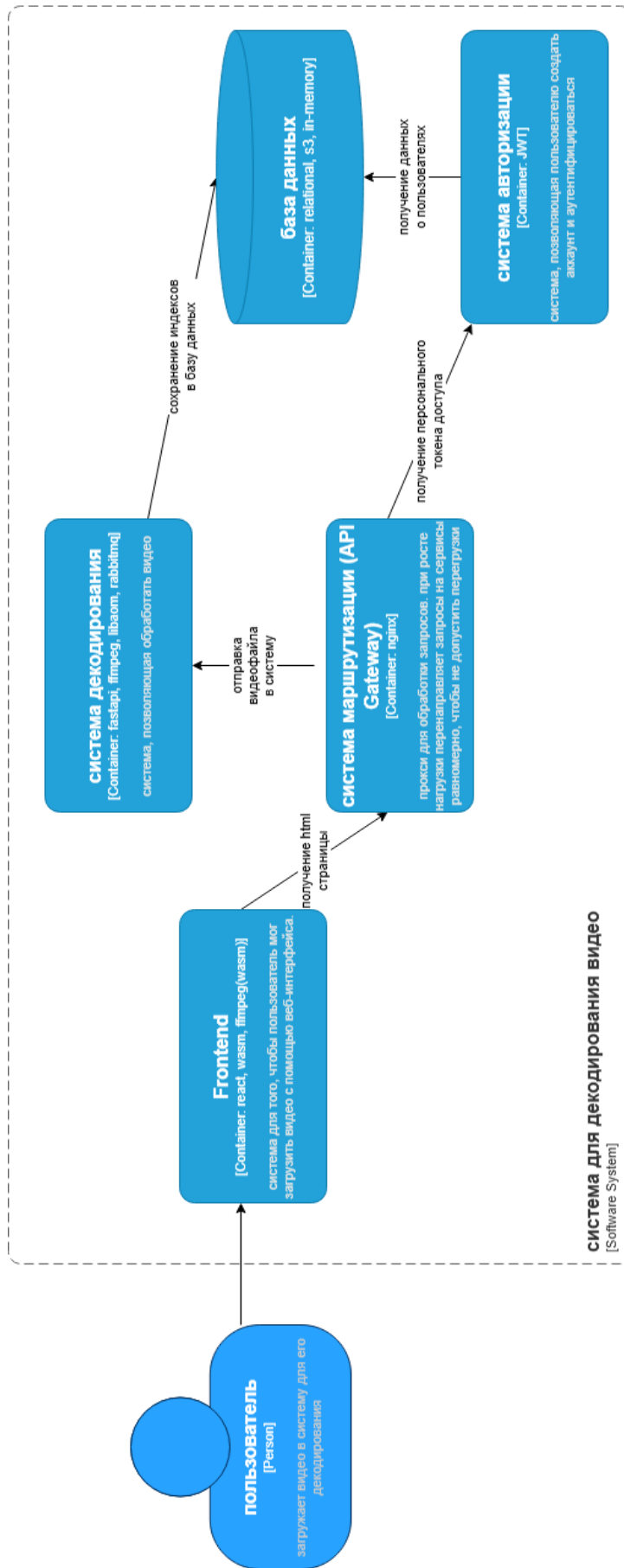


Рисунок 16. диаграмма контейнеров системы.

Диаграмма компонентов представлена на рис. 17. Рассмотрим самые важные из них в следующих главах.

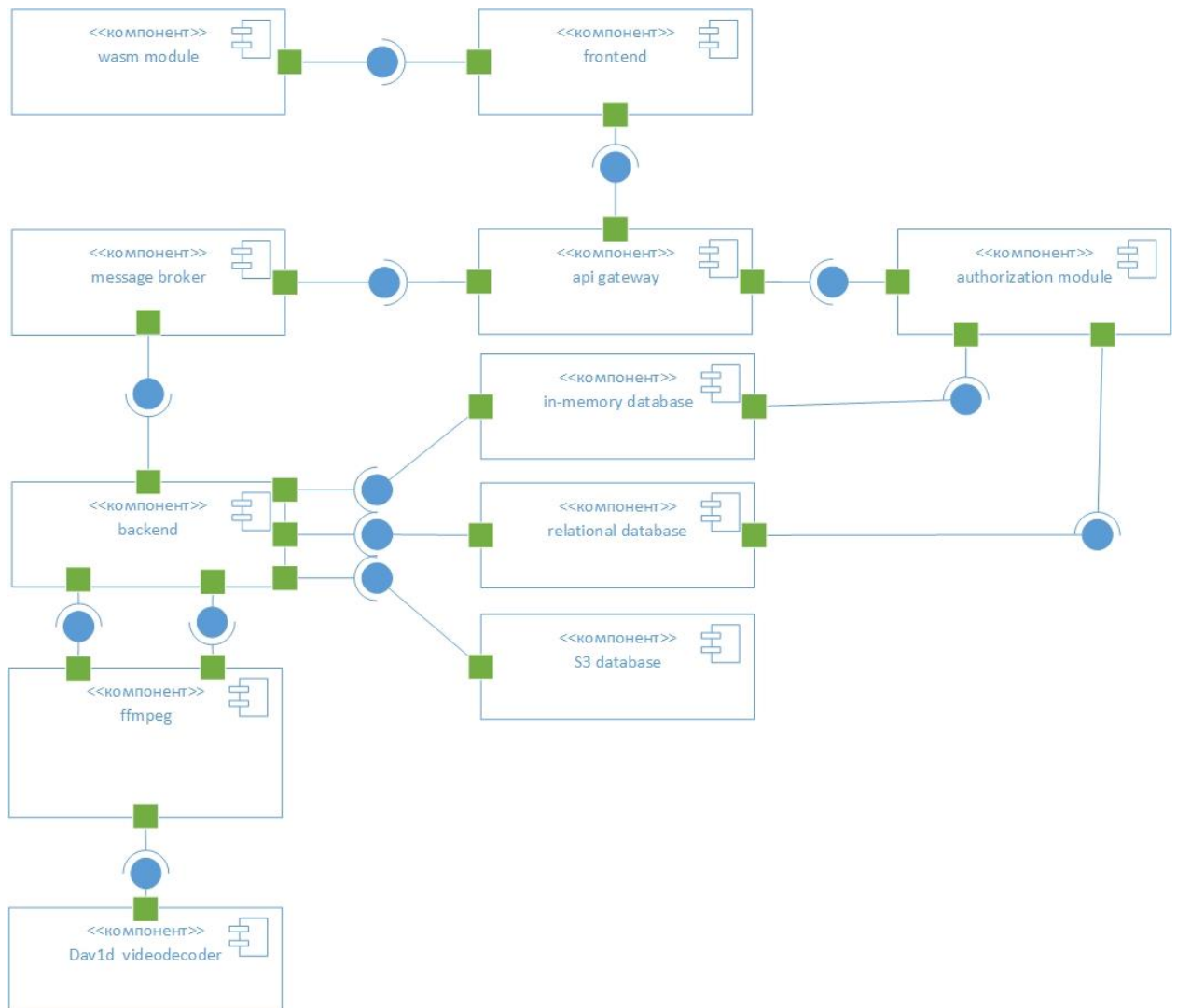


Рисунок 17. диаграмма компонентов оптимизированной системы

2.2.1. Проектирование реляционной базы данных

Для данной системы будут использоваться несколько различных баз: low-latency база для горячих данных, реляционная база для стандартных реляционных данных и S3 хранилище Serp для видеофайлов и индексов.

Опустим проектирование нереляционных баз ввиду того, что их структура легче поддаётся изменению.

В качестве реляционной базы будет использован PostgreSQL ввиду его масштабируемости и интеграции, а также большей устойчивостью к высоким нагрузкам чем у его прямого конкурента MySQL.

Структура таблиц будет перенесена из существующей базы, ER диаграмма которой представлена на рис. 12, так что она будет оставлена без изменений. Имеет смысл рассмотреть предназначение таблиц в существующей базе данных.

Таблица Users, как и в наивной реализации, хранит данные о пользователях, но в гораздо большем объеме, включая метод авторизации, дату регистрации, тарифный план и т.д.

Таблица Roles и соответствующая ей таблица r_users_roles позволяют установить соответствие конкретного пользователя/пользователей к роли/ролям, будь то администратор или обычный пользователь.

Таблицы Videos и Api_keys хранят информацию, аналогичную тем же таблицам в наивной реализации.

2.2.2. Проектирование бэкенда

Бэкенд системы является центральным компонентом всей архитектуры и отвечает за обработку пользовательских запросов, управление задачами декодирования, взаимодействие с базами данных и интеграцию с видеodeкодером. Основной задачей серверной части является обеспечение максимально быстрой обработки операций «загрузка – декодирование» при сохранении масштабируемости системы. Для этого был выбран подход горизонтального масштабирования, при котором увеличение вычислительных мощностей достигается за счет увеличения серверов. Компонент Api Gateway проксирует запросы на свободный сервер-реплику с компонентами бэкенда и декодера, а если такового нет, то на наименее занятый сервер. Степень загрузки мониторится из бэкенда и сохраняется в redis.

Для того, чтобы обработать все запросы, будет применен паттерн «request queue», реализация которого в архитектуре системы представлена на рис. 18. Смысл данного паттерна в том, чтобы сохранять запросы в очередь и равномерно маршрутизировать их на реплики бэкенда, что гарантирует обработку всех запросов.

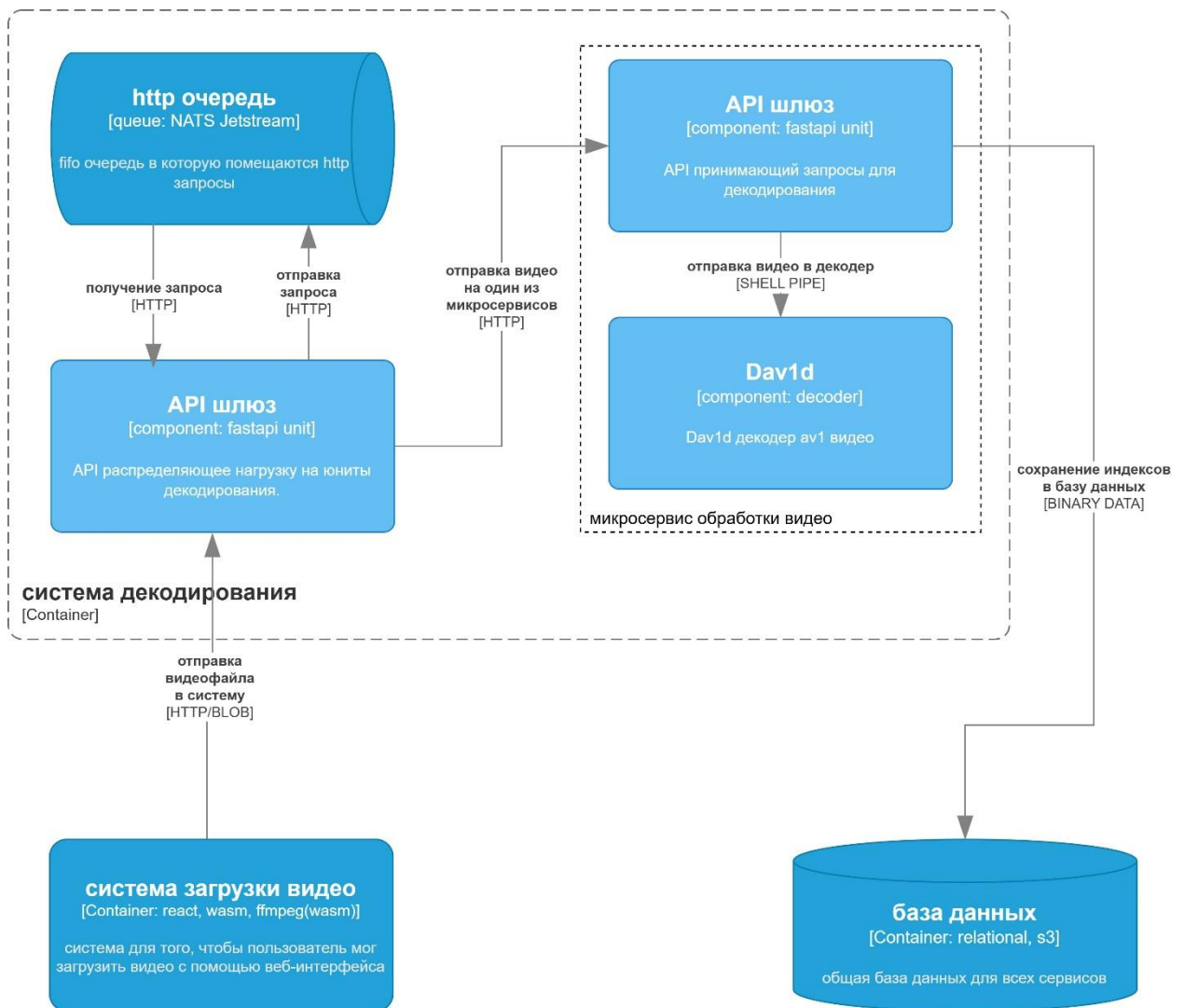


Рисунок 18. Схема системы декодирования

Как видно из рис. 18, бэкенд связан с компонентами message broker, api gateway и ffmpeg. FFMPEG является инструментом с открытым кодом для работы с видео. В нём реализованы методы работы с файловой системой и видеodeкодерами, поэтому и в наивной, и в оптимизированной реализации он будет необходим для использования Dav1d. Коммуникация с FFMPEG будет происходить через shell, с использованием in-memory файловой системы или S3 хранилища когда это невозможно. На рис. 19 представлена упрощенная реализация decision tree маршрутизатора. Наиболее приоритетный сервер для декодирования – сервер со свободной оперативной памятью и вычислительными мощностями. В случае, если не хватает оперативной памяти, будет использован метод потокового декодирования из S3 хранилища. На данный при определении доступной вычислительной мощности для декодирования исходим из того, что количество доступных ядер =

(суммарное количество логических ядер процессора – 1), так как одно из ядер должно оставаться за wsgi сервером. В случае превышения этого количества ожидается снижение производительности за счет вытеснения одного процесса другим. В главе «реализация» будет экспериментально проверено оптимальное количество ядер.

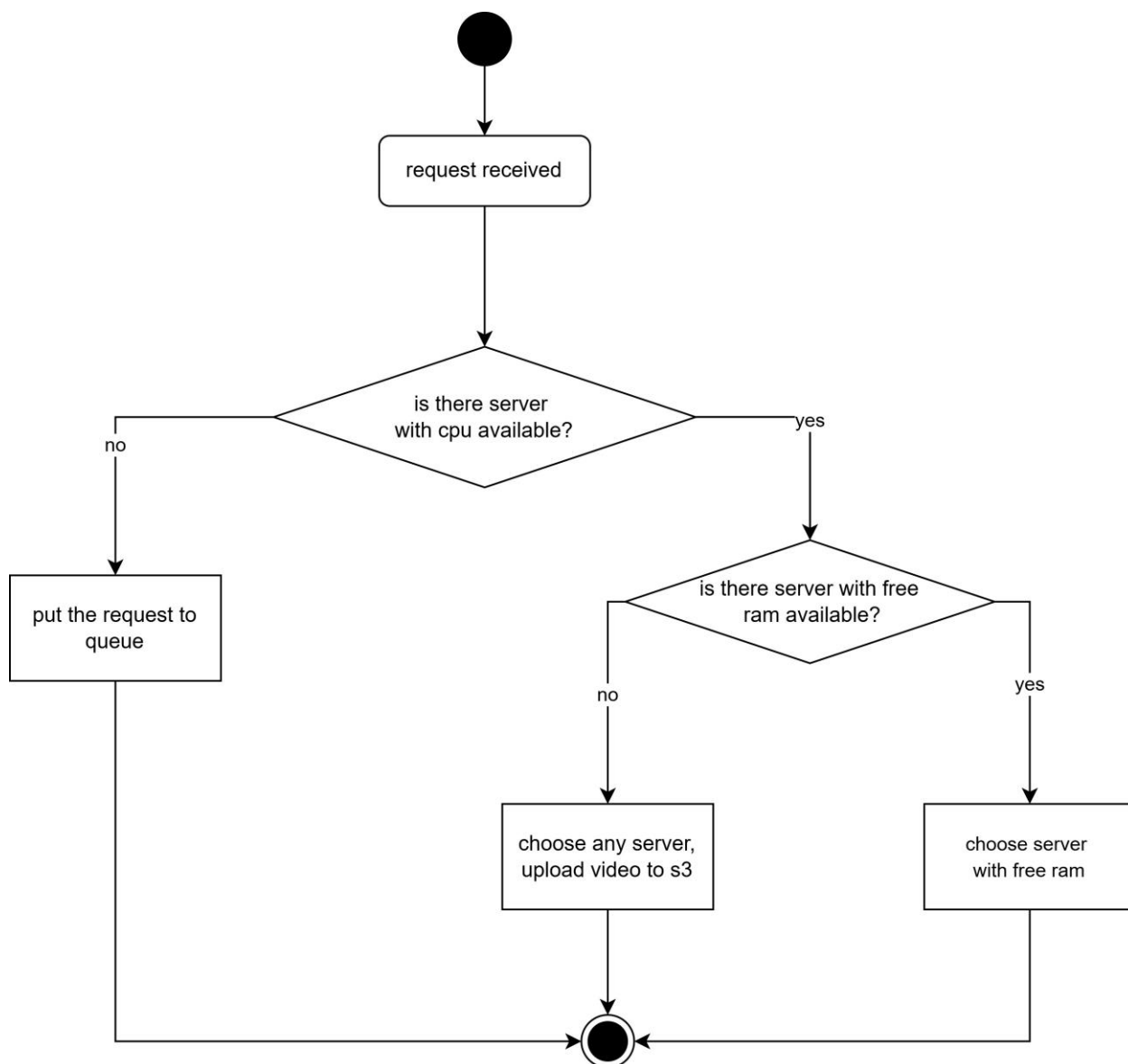


Рисунок 19. Алгоритм метода выбора сервера для декодирования

Как было рассмотрено в главе 2.2.1, каждому видео соответствует статус обработки. Это целое значение, которое имеет несколько состояний: uploaded, queued, processing, done, failed, retrying. Их возможная смена показана на рис. 20.

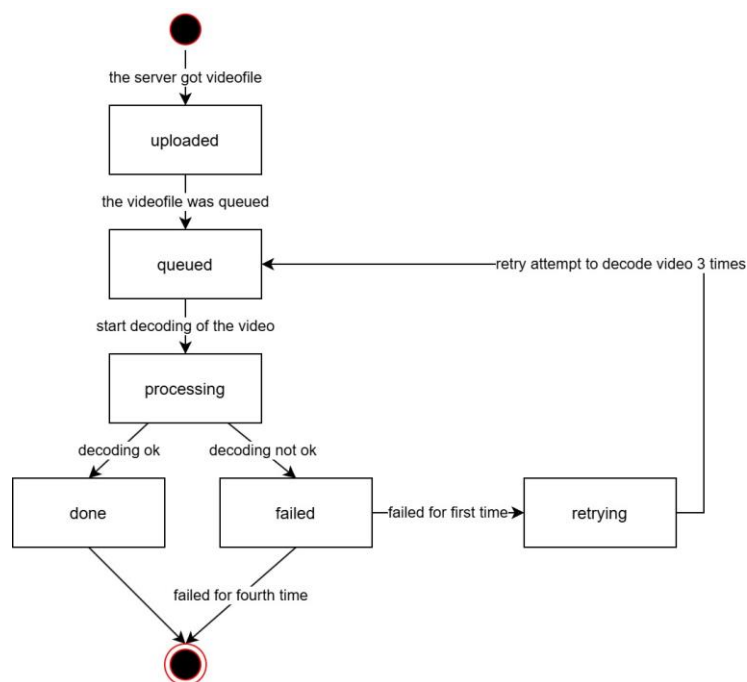


Рисунок 20. диаграмма состояний видео

В таблице 5 показаны требуемые REST эндпоинты, которые будут реализованы в API.

Таблица 5 – требуемые эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ	Код
POST	/auth/register	Регистрация пользователя	Phone, password	user_id	201
POST	/auth/login	Авторизация	Phone, password	access_token, refresh_token	200
POST	/auth/refresh	Обновление токена	Refresh_token	access_token	204
POST	/auth/logout	Выход	-		200
POST	/archives	Создать архив	Name	archive_id	200
GET	/archives	Список архивов пользователя		[{archive_id, name, created_at}]	200
POST	/archives/{archive_id}/videos	Загрузить видео в архив	.obu файл	video_id, task_id	202
GET	/archives/{archive_id}/videos	Список видео в архиве		[{video_id, name, state}]	200

DELETE	/videos/{video_id}	Удалить видео			204
--------	--------------------	---------------	--	--	-----

Выше было упомянуто, что задача имеет свой статус, однако в требуемых эндпоинтах нет опции узнать статус. Это сделано из-за того, что нотификация пользователя будет происходить с помощью соединения через вебсокет.

Уведомление через вебсокет – довольно удобное и логичное решение, избавляющее от необходимости посылать http запрос каждый несколько секунд чтобы узнать статус. Однако появляется новое ограничение, связанное с особенностью вебсокета: при большом количестве подключений система начинает работать медленнее. Чтобы избежать замедления воспользуемся мощностями серверов, проксируемых с помощью API Gateway. Каждое соединение вебсокета будет держаться на отдельном сервере до момента пока у пользователя есть активные задачи. Когда приходит уведомление о конце задачи в очередь происходит бродкаст-рассылка этого сообщения на все реплики, и реплика, выполняющая эту задачу, закрывает соединение. Это показано на рис. 21.

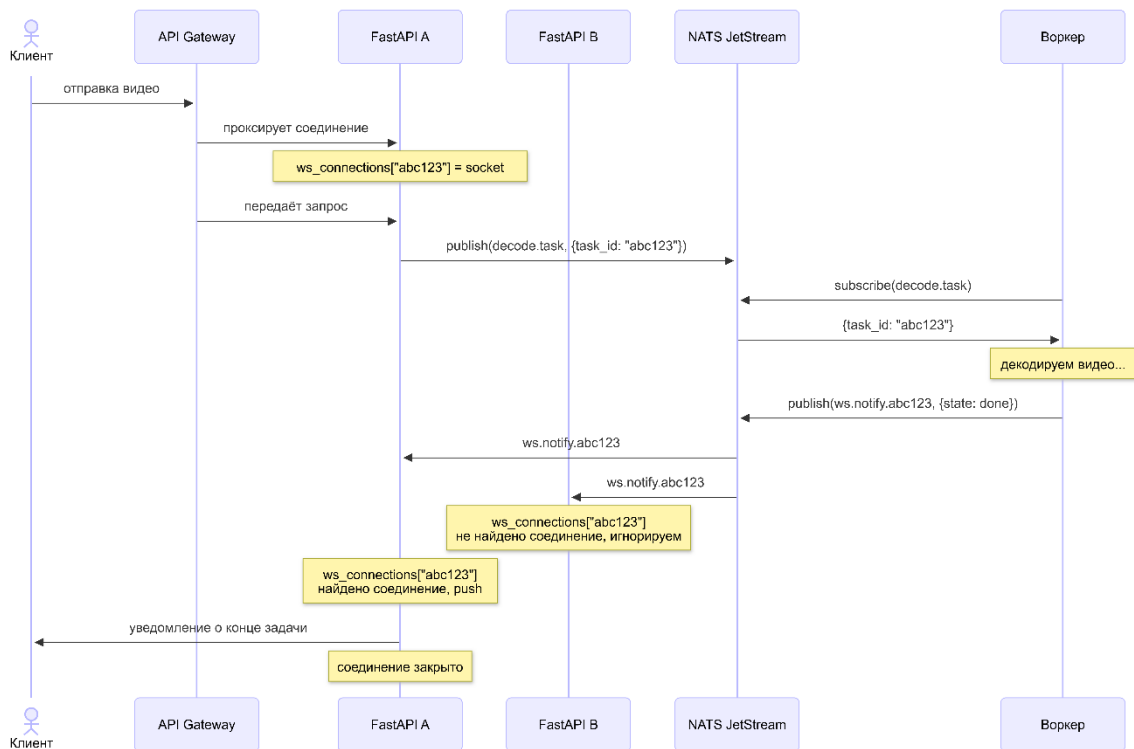


Рисунок 21. диаграмма последовательности для уведомления клиента

2.2.3. Проектирование фронтенда

Фронтенд системы выполняет несколько задач, включающих в себя авторизацию пользователя, загрузку видеофайлов, их предварительную обработку и отправку обработанных видео на сервер.

Для отделения аудиоданных от видео используется FFMPEG, скомпилированный под архитектуру WebAssembly. Компиляция осуществляется с помощью инструмента Emscripten, что позволит запустить код на C в браузере, на стороне клиента.

Дизайн не играет особой роли в данной системе (по крайней мере, не является основной целью), так что было принято решение спроектировать его максимально простым. Интерфейс состоит из трех экранов: регистрация, логин и загрузка видео. Их прототипы представлены на рис. 22, 23 и 24 соответственно.

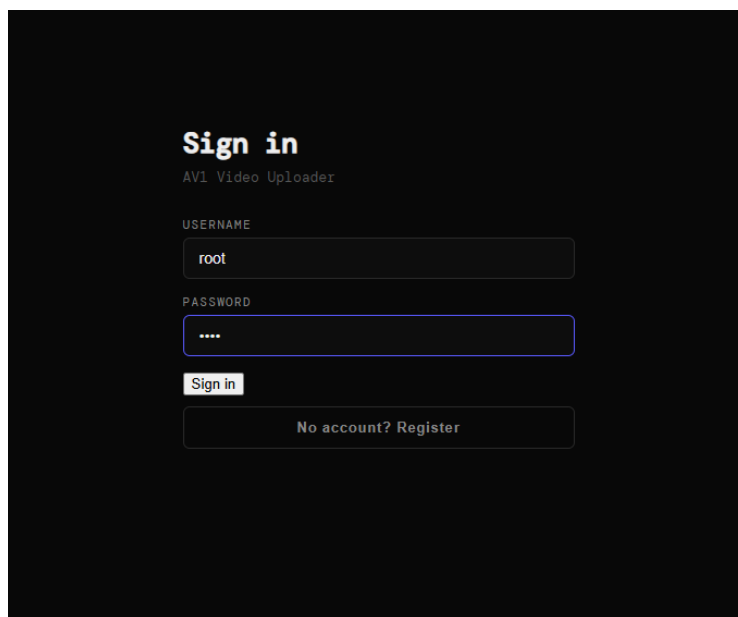


Рисунок 22. экран входа

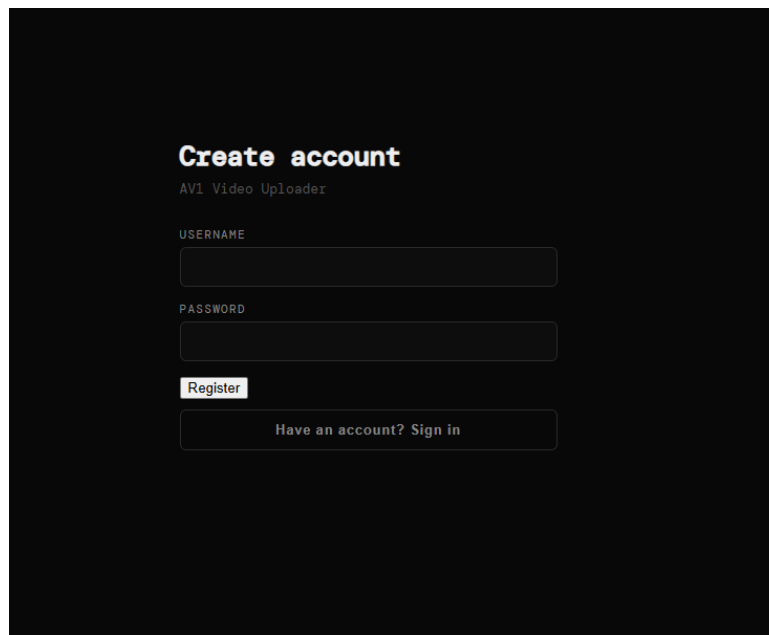


Рисунок 23. экран регистрации

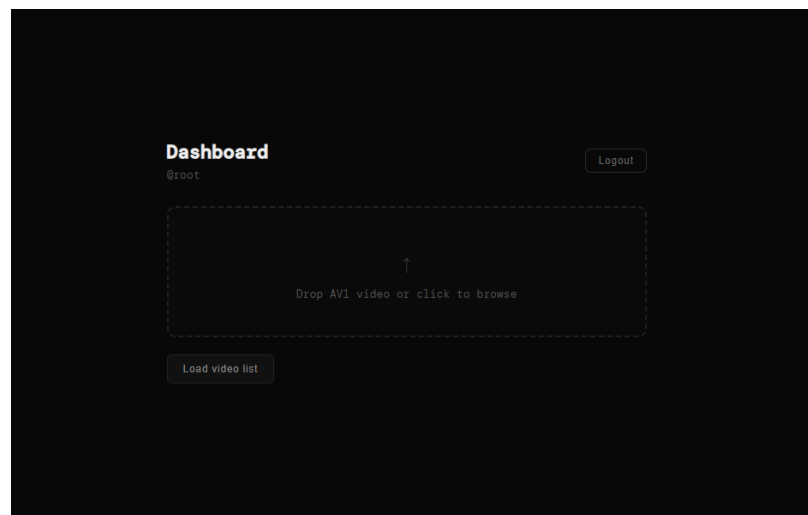


Рисунок 24. экран загрузки видео

2.2.4. Проектирование оптимизаций Dav1d

В данной главе будут описаны основные методы оптимизации Dav1d, ранее затронутые в главе 1. Стоит оговориться, что видеodeкодер — крайне оптимизированное ПО, и любое серьёзное вмешательство может вызвать в нём эффект домино, сбивая общую структуру, оптимизированную под процессорный кеш, архитектуру процессора и пр. В связи с этим в данной главе не будут предлагаться радикальные вмешательства, меняющие общую структуру данных или логику работы кодера.

Технология CDEF

Constrained directional enhancement filter, как было рассмотрено в разделе 1.7.2, является ресурсоемкой но важной частью для сохранения качества видео при декодировании. Тем не менее, в задаче декодирования в сверхнизком разрешении артефакты на границах блоков, как правило, не влияют на качество видео, так что будут проведены два эксперимента: с четырьмя фильтрами с углами наклона (45° , 0° , -45° , -90°) вместо восьми исходных (45° , 22.5° , 0° , -22.5° , -45° , -67.5° , -90° , -112.5°) и с отключённым CDEF.

Технология DF

В отличие от CDEF, DF имеет также высокое значение для низкого разрешения. Это значит, что если отключить фильтр блочности сразу на всех кадрах, то артефакты будут накапливаться от опорных кадров с каждым новым В-кадром. Для устранения этого эффекта будет использоваться подход, при котором фильтр блочности будет применяться только для опорных кадров, хоть стандарт errata и подразумевает что DF должен быть включен либо выключен везде. В таком случае накопление артефактов должно замедлиться, учитывая, что каждый кадр использует сразу несколько опорных кадров. Также будет реализована версия, в которой DF вырезан полностью, чтобы подтвердить или опровергнуть вышеупомянутую гипотезу.

Энтропийное декодирование

Теоретически энтропийное декодирование занимает меньше ресурсов, чем интерполяция или фильтры. Тем не менее, оно происходит на каждом кадре в минимум одном блоке, так что имеет смысл попробовать его оптимизировать.

Как упоминалось в главе анализа, DCT и IDCT использует таблицу частот, также называемую таблицей квантизации. В качестве оптимизации можно применить подход обрезки, квантовав её в 4 раза, оставив только частоты в верхнем левом квадрате. На рис. 25 красным цветом обозначены частоты, которые будут оставлены, а синим – все доступные частоты при операции IDCT. Если гипотеза верна, то при такой оптимизации видео с низким разрешением почти не потеряет качество, так как высокие частоты используются для кодирования мелких деталей видео, которые не важны. В случае успеха оптимизация будет расширена до других методов энтропийного декодирования.

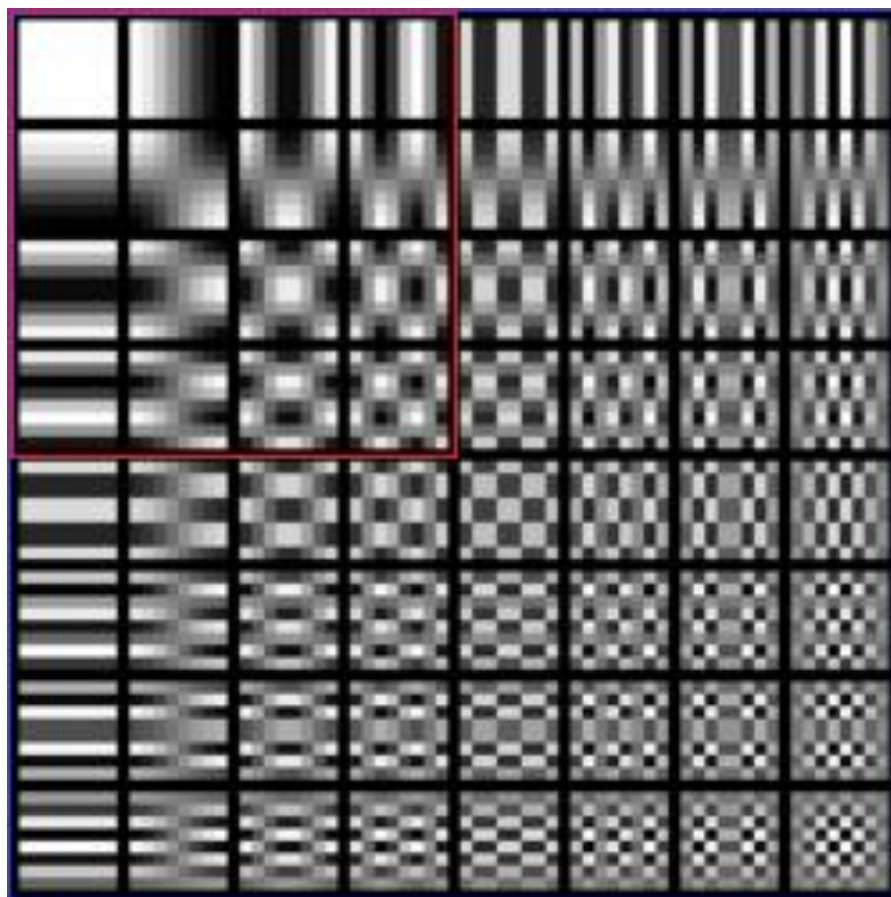


Рисунок 25. объяснение метода квантизации таблицы коэффициентов

Интерполяция

Для оптимизации интерполяции, как и для DF и CDEF будут использоваться 2 метода: полный вырез и максимальное облегчение. Для второго варианта нужно предусмотреть сохранение качества, потому что иначе может произойти каскадная деградация качества. На рис. 26 можно заметить, что в процессе интерполяции коэффициенты на 1,2,7 и 8 позициях очень редко имеют большое значение. Это значит, что возможно попробовать без ущерба для качества видео вырезать их и оставить 4-tap интерполяцию по коэффициентам 3-6, либо 2-tap, в зависимости от метрики PSNR по результату.

Position	Regular	Smooth	Sharp	Bilinear	4-Tap Regular	4-Tap Smooth
0/16	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}
1/16	{0,2,-6,126,8,-2,0,0}	{0,2,28,62,34,2,0,0}	{-2,2,-6,126,8,-2,2,0}	{0,0,0,120,8,0,0,0}	{0,0,-4,126,8,-2,0,0}	{0,0,30,62,34,2,0,0}
2/16	{0,2,-10,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}	{-2,6,-12,124,16,-6,4,-2}	{0,0,0,112,16,0,0,0}	{0,0,-8,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}
3/16	{0,2,-12,116,28,-8,2,0}	{0,0,22,62,40,4,0,0}	{-2,8,-18,120,26,-10,6,-2}	{0,0,0,104,24,0,0,0}	{0,0,-10,116,28,-6,0,0}	{0,0,22,62,40,4,0,0}
4/16	{0,2,-14,110,38,-10,2,0}	{0,0,20,60,42,6,0,0}	{-4,10,-22,116,38,-14,6,-2}	{0,0,0,96,32,0,0,0}	{0,0,-10,110,38,-8,0,0}	{0,0,20,60,42,6,0,0}
5/16	{0,2,-14,102,48,-12,2,0}	{0,0,18,58,44,8,0,0}	{-4,10,-22,108,48,-18,8,-2}	{0,0,0,88,40,0,0,0}	{0,0,-12,102,48,-10,0,0}	{0,0,18,58,44,8,0,0}
6/16	{0,2,-16,94,58,-12,2,0}	{0,0,16,56,46,10,0,0}	{-4,10,-24,100,60,-20,8,-2}	{0,0,0,80,48,0,0,0}	{0,0,-14,94,58,-10,0,0}	{0,0,16,56,46,10,0,0}
7/16	{0,2,-14,84,66,-12,2,0}	{0,-2,16,54,48,12,0,0}	{-4,10,-24,90,70,-22,10,-2}	{0,0,0,72,56,0,0,0}	{0,0,-12,84,66,-10,0,0}	{0,0,14,54,48,12,0,0}
8/16	{0,2,-14,76,76,-14,2,0}	{0,-2,14,52,52,14,-2,0}	{-4,12,-24,80,80,-24,12,-4}	{0,0,0,64,64,0,0,0}	{0,0,-12,76,76,-12,0,0}	{0,0,12,52,52,12,0,0}
9/16	{0,2,-12,66,84,-14,2,0}	{0,0,12,48,54,16,-2,0}	{-2,10,-22,70,90,-24,10,-4}	{0,0,0,56,72,0,0,0}	{0,0,-10,66,84,-12,0,0}	{0,0,12,48,54,14,0,0}
10/16	{0,2,-12,58,94,-16,2,0}	{0,0,10,46,56,16,0,0}	{-2,8,-20,60,100,-24,10,-4}	{0,0,0,48,80,0,0,0}	{0,0,-10,58,94,-14,0,0}	{0,0,10,46,56,16,0,0}
11/16	{0,2,-12,48,102,-14,2,0}	{0,0,8,44,58,18,0,0}	{-2,8,-18,48,108,-22,10,-4}	{0,0,0,40,88,0,0,0}	{0,0,-10,48,102,-12,0,0}	{0,0,8,44,58,18,0,0}
12/16	{0,2,-10,38,110,-14,2,0}	{0,0,6,42,60,20,0,0}	{-2,6,-14,38,116,-22,10,-4}	{0,0,0,32,96,0,0,0}	{0,0,-8,38,110,-12,0,0}	{0,0,6,42,60,20,0,0}
13/16	{0,2,-8,28,116,-12,2,0}	{0,0,4,40,62,22,0,0}	{-2,6,-10,26,120,-18,8,-2}	{0,0,0,24,104,0,0,0}	{0,0,-6,28,116,-10,0,0}	{0,0,4,40,62,22,0,0}
14/16	{0,0,-4,18,122,-10,2,0}	{0,0,4,36,62,26,0,0}	{-2,4,-6,16,124,-12,6,-2}	{0,0,0,16,112,0,0,0}	{0,0,-4,18,122,-8,0,0}	{0,0,4,36,62,26,0,0}
15/16	{0,0,-2,8,126,-6,2,0}	{0,0,2,34,62,28,2,0}	{0,2,-2,8,126,-6,2,-2}	{0,0,0,8,120,0,0,0}	{0,0,-2,8,126,-4,0,0}	{0,0,2,34,62,30,0,0}

Рисунок 26. таблица коэффициентов интерполяции [11]

2.3. Выводы

В ходе проектирования была разработана архитектура системы быстрого декодирования видео в формате AV1, удовлетворяющая функциональным и нефункциональным требованиям, сформулированным в главе 1.

Для обеспечения возможности сравнительного анализа была спроектирована наивная реализация системы на основе монолитного FastApi-приложения с хранилищем SQLite. Данная реализация намеренно лишена каких-либо оптимизаций и служит базовой точкой отсчёта для измерения эффективности оптимизированной версии.

Оптимизированная реализация системы построена на принципах горизонтального масштабирования и минимизации задержек на каждом уровне стека. В качестве ключевых архитектурных решений были приняты следующие: использование брокера сообщений NATS JetStream для асинхронной маршрутизации задач декодирования между репликами бэкенда; применение Redis для хранения горячих данных о состоянии задач с целью снижения задержки доступа по сравнению с реляционной базой данных; декодирование видео из оперативной памяти в приоритетном порядке, с использованием S3-хранилища Serp в качестве запасного варианта при недостатке памяти.

Спроектирован REST API, необходимый для авторизации, управления архивами и жизненного цикла задач декодирования. Для уведомления клиента о завершении декодирования выбран протокол WebSocket вместо HTTP-поллинга, что исключает избыточные запросы к серверу и снижает итоговую задержку уведомления с 0–2000 мс до уровня сетевой латентности.

Жизненный цикл задачи декодирования формализован в виде диаграммы состояний, включающей шесть состояний: `uploaded`, `queued`, `processing`, `done`, `failed`, `retrying`.

На стороне клиента спроектирован модуль предварительной обработки видео на базе `ffmpeg`, скомпилированного в `WebAssembly`, выполняющий отделение аудиодорожки перед отправкой файла на сервер. Данное решение позволяет сократить объём передаваемых данных без потери информации, необходимой для индексации.

Таким образом, спроектированная система обеспечивает выполнение всех требований, выявленных в ходе анализа предметной области, и формирует основу для последующей реализации и оптимизации декодера `Dav1d`, описанных в главе 3.

ГЛАВА 3. РЕАЛИЗАЦИЯ ОПТИМИЗАЦИЙ И СИСТЕМ ДЕКОДИРОВАНИЯ

В данной главе совмещены написание финальной оптимизированной системы, оптимизации DAV1D и тестирование оптимизаций с указанием их влияния на итоговую производительность.

Глава состоит из четырёх разделов: «оптимизации DAV1D», «финальная система», «тестирование», «исходный код». В первом разделе описаны проведённые оптимизации DAV1D и их результаты. Во втором разделе описана реализация оптимизированной и наивной систем. В третьем разделе описаны протоколы тестирования каждой из частей и их результат. В четвёртом приведены ссылки на репозитории с работой.

3.1. Оптимизации DAV1D

Все оптимизации, приведённые ниже, протестированы с помощью утилиты Hyperfine и тестового отрывка фильма «Fast & furious 8» длиной в 30 минут, каждая оптимизация тестируется в 3 итерации с прогревом жесткого диска в одноядерном режиме, в качестве оценки используется минимальное время, так как оно является минимально зашумленной метрикой. Тесты запущены на сервере с ОС ubuntu 22.04.5, без графической оболочки и устройств пользовательского ввода, без других запущенных пользовательских процессов кроме родительского процесса бенчмарка и дочернего процесса декодирования.

Для каждой оптимизации также представлено визуальное сравнение с помощью утилиты FFMPEG.

Минимальное время, замеренное на стандартной имплементации DAV1D с отключенными SIMD, в однопоточном режиме: 545.83с.

3.1.1. CDEF, Loop filter (DF)

Вопреки ожиданиям, отключение одновременно Constrained Directional Enhancement Filter и Deblocking Filter (называющийся в стандарте errata Loop Filter) снизило качество на незначительные значения, при этом существенно сократив время обработки видео на 26%, составив 404с со средним значением 415с. В связи с этим нет необходимости производить оптимизации данных частей. Тем не менее,

представленные методы могут быть использованы при оптимизации видео для более высокого разрешения.

<pre>37 static NOINLINE void 38 loop_filter(pixel *dst, int E, int I, int H, 39 const ptrdiff_t stridea, const 40 ptrdiff_t strideb, const int wd 41 HIGHBD_DECL_SUFFIX) 42 { 43 const int bitdepth_min_8 = bitdepth_from_max 44 (bitdepth_max) - 8; 45 const int F = 1 << bitdepth_min_8; 46 E <= bitdepth_min_8; 47 I <= bitdepth_min_8; 48 H <= bitdepth_min_8;</pre>	<pre>37 static NOINLINE void 38 loop_filter(pixel *dst, int E, int I, int H, 39 const ptrdiff_t stridea, const 40 ptrdiff_t strideb, const int wd 41 HIGHBD_DECL_SUFFIX) 42 { 43 return; 44 const int bitdepth_min_8 = bitdepth_from_max 45 (bitdepth_max) - 8; 46 const int F = 1 << bitdepth_min_8; 47 E <= bitdepth_min_8; 48 I <= bitdepth_min_8; 49 H <= bitdepth_min_8;</pre>
---	--

Рисунок 27. дельта файла *loopfilter_tmpl.c*

<pre>71 COLD void dav1d_default_settings(Dav1dSettings 72 *const s) { 73 s->n_threads = 0; 74 s->max_frame_delay = 0; 75 s->apply_grain = 1; 76 s->allocator.cookie = NULL; 77 s->allocator.alloc_picture_callback = 78 dav1d_default_picture_alloc; 79 s->allocator.release_picture_callback = 80 dav1d_default_picture_release; 81 s->logger.cookie = NULL; 82 s->logger.callback = 83 dav1d_log_default_callback; 84 s->operating_point = 0; 85 s->all_layers = 1; // just until the tests 86 are adjusted 87 s->frame_size_limit = 0; 88 s->strict_std_compliance = 0; 89 s->output_invisible_frames = 0; 90 s->inloop_filters = DAV1D_INLOOPFILTER_ALL; 91 s->decode_frame_type = 92 DAV1D_DECODEFRAMETYPE_ALL;</pre>	<pre>71 COLD void dav1d_default_settings(Dav1dSettings 72 *const s) { 73 s->n_threads = 0; 74 s->max_frame_delay = 0; 75 s->apply_grain = 1; 76 s->allocator.cookie = NULL; 77 s->allocator.alloc_picture_callback = 78 dav1d_default_picture_alloc; 79 s->allocator.release_picture_callback = 80 dav1d_default_picture_release; 81 s->logger.cookie = NULL; 82 s->logger.callback = 83 dav1d_log_default_callback; 84 s->operating_point = 0; 85 s->all_layers = 1; // just until the tests 86 are adjusted 87 s->frame_size_limit = 0; 88 s->strict_std_compliance = 0; 89 s->output_invisible_frames = 0; 90 s->inloop_filters = DAV1D_INLOOPFILTER_NONE; 91 s->decode_frame_type = 92 DAV1D_DECODEFRAMETYPE_ALL;</pre>
---	--

Рисунок 28. дельта файла *lib.c*

3.1.2. Интерполяция

Для оптимизации одного из самых ресурсоемких процессов было использовано несколько методов, каждый из них приведён ниже.

3.1.2.1. Сокращение количества фильтров до 6

В качестве первого метода оптимизации интерполяции был выбран метод сокращения фильтров интерполяции с восьми до шести, что сократило время декодирования на 30.5%, составив 379с со средним значением 393с. Данная оптимизация не затрагивает коэффициенты интерполяции, представленные на рис. 26, и сохраняет приемлемое качество видео ввиду того, что коэффициенты на позициях 1 и 8 (опущенные в данном варианте), как правило, имеют значение 0, то есть не влияют на итоговый результат.

<pre> 77 #define FILTER_8TAP(src, x, F, stride) \ 78- (F[0] * src[x + -3 * stride] + \ 79- F[1] * src[x + -2 * stride] + \ 80- F[2] * src[x + -1 * stride] + \ 81- F[3] * src[x + +0 * stride] + \ 82- F[4] * src[x + +1 * stride] + \ 83- F[5] * src[x + +2 * stride] + \ 84- F[6] * src[x + +3 * stride] + \ 85- F[7] * src[x + +4 * stride]) 86 87 #define FILTER_8TAP2(src, x, F) \ 88- (F[0] * src[0][x] + \ 89- F[1] * src[1][x] + \ 90- F[2] * src[2][x] + \ 91- F[3] * src[3][x] + \ 92- F[4] * src[4][x] + \ 93- F[5] * src[5][x] + \ 94- F[6] * src[6][x] + \ 95- F[7] * src[7][x]) 86 </pre>	<pre> 77 #define FILTER_8TAP(src, x, F, stride) \ 78+ (\ 79- F[1] * src[x + -2 * stride] + \ 80- F[2] * src[x + -1 * stride] + \ 81- F[3] * src[x + +0 * stride] + \ 82- F[4] * src[x + +1 * stride] + \ 83- F[5] * src[x + +2 * stride] + \ 84+ F[6] * src[x + +3 * stride] \ 85+) 86 87 #define FILTER_8TAP2(src, x, F) \ 88+ (\ 89- F[1] * src[1][x] + \ 90- F[2] * src[2][x] + \ 91- F[3] * src[3][x] + \ 92- F[4] * src[4][x] + \ 93- F[5] * src[5][x] + \ 94+ F[6] * src[6][x] \ 95+) 86 </pre>
---	---

Рисунок 29. дельта файла *mc_tmpl.c*

3.1.2.2. Сокращение вариантов интерполяции до 6

В качестве второго метода оптимизации были сокращены варианты интерполяции с 90 до 6. Каждый из вариантов, представленный на рис. 26, имеет смещение к левому краю или правому, сохраняя баланс коэффициентов. Возьмём, к примеру, коэффициенты *sharp* 0/16 и 10/16, представляющие собой {0,0,0,128,0,0,0,0} и {-2,8,-20,60,100,-24,10,-4}. Как мы можем видеть, коэффициенты дают одинаковую сумму, но в 10/16 сумма распределена влево, хотя фильтр тот же. Данная оптимизация убирает вариацию между различными фильтрами интерполяции, жестко фиксируя фильтр на 7/16, то есть промежуточном значении с нейтральным смещением. Это оптимизирует локальность данных и позволяет закешировать все фильтры интерполяции в одну линию процессорного кеша, в случае если его размер для этого подходит. Для этого также структура фильтров выравнивается в памяти по 64 байтам, что достаточно для всей структуры. Результат оптимизации: 32.5%, 368с со средним значением 384с.

<pre> 115 #define GET_H_FILTER(mx) \ 116- const int8_t *const fh = !(mx) ? \ 117- NULL : w > 4 ? \ 118- david_mc_subpel_filters \ 119- [filter_type & 3][(mx) - 1] : \ 120- david_mc_subpel_filters[3 + \ 121- (filter_type & 1)][(mx) - 1] 122 123 #define GET_V_FILTER(my) \ 124- const int8_t *const fv = !(my) ? \ 125- NULL : h > 4 ? \ 126- david_mc_subpel_filters \ 127- [filter_type >> 2][(my) - 1] : \ 128- david_mc_subpel_filters[3 + \ 129- ((filter_type >> 2) & 1)][(my) - \ 130- 1] 124 </pre>	<pre> 115 #define GET_H_FILTER(mx) \ 116- const int8_t *const fh = !(mx) ? \ 117- NULL : w > 4 ? \ 118- david_mc_subpel_filters \ 119- [filter_type & 3][0] : \ 120- david_mc_subpel_filters[3 + \ 121- (filter_type & 1)][0] 122 123 #define GET_V_FILTER(my) \ 124- const int8_t *const fv = !(my) ? \ 125- NULL : h > 4 ? \ 126- david_mc_subpel_filters \ 127- [filter_type >> 2][0] : \ 128- david_mc_subpel_filters[3 + \ 129- ((filter_type >> 2) & 1)][0] 124 </pre>
---	--

Рисунок 30. дельта файла *mc_tmpl.c*

```

444 ATTR_MCMODEL_SMALL
445 const int8_t ALIGN(david_mc_subpel_filters[6][15][0], 8) = {
446     [DAVID_FILTER_8TAP_REGULAR] = {
447         { 0, 1, -3, 63, 4, -1, 0, 0 },
448         { 0, 1, -5, 61, 9, -2, 0, 0 },
449         { 0, 1, -6, 58, 14, -4, 1, 0 },
450         { 0, 1, -7, 55, 19, -5, 1, 0 },
451         { 0, 1, -7, 51, 24, -6, 1, 0 },
452         { 0, 1, -8, 47, 29, -6, 1, 0 },
453         { 0, 1, -7, 42, 33, -6, 1, 0 },
454         { 0, 1, -7, 38, 38, -7, 1, 0 },
455         { 0, 1, -6, 33, 42, -7, 1, 0 },
456         { 0, 1, -6, 29, 47, -8, 1, 0 },
457         { 0, 1, -6, 24, 51, -7, 1, 0 },
458         { 0, 1, -5, 19, 55, -7, 1, 0 },
459         { 0, 1, -4, 14, 58, -6, 1, 0 },
460         { 0, 0, -2, 9, 61, -5, 1, 0 },
461         { 0, 0, -1, 4, 63, -3, 1, 0 },
462     }, [DAVID_FILTER_8TAP_SMOOTH] = {
463         { 0, 1, 14, 31, 17, 1, 0, 0 },
464         { 0, 0, 13, 31, 18, 2, 0, 0 },
465         { 0, 0, 11, 31, 20, 2, 0, 0 },
466         { 0, 0, 10, 30, 21, 3, 0, 0 },
467         { 0, 0, 9, 29, 22, 4, 0, 0 },
468         { 0, 0, 8, 28, 23, 5, 0, 0 },
469         { 0, -1, 8, 27, 24, 6, 0, 0 },
470         { 0, -1, 7, 26, 26, 7, -1, 0 },
471         { 0, 0, 6, 24, 27, 8, -1, 0 },
472         { 0, 0, 5, 23, 28, 8, 0, 0 },
473         { 0, 0, 4, 22, 29, 9, 0, 0 },
474         { 0, 0, 3, 21, 30, 10, 0, 0 },
475         { 0, 0, 2, 20, 31, 11, 0, 0 },
476         { 0, 0, 2, 18, 31, 13, 0, 0 },
477         { 0, 0, 1, 17, 31, 14, 1, 0 },
478     }, [DAVID_FILTER_8TAP_SHARP] = {
479         { -1, 1, -3, 63, 4, -1, 1, 0 },
480         { -1, 3, -6, 62, 8, -3, 2, -1 },
481         { -1, 4, -9, 60, 13, -5, 3, -1 },
482         { -2, 5, -11, 58, 19, -7, 3, -1 },
483         { -2, 5, -11, 54, 24, -9, 4, -1 },
484         { -2, 5, -12, 50, 30, -10, 4, -1 },
485         { -2, 5, -12, 45, 35, -11, 5, -1 },
486         { -2, 6, -12, 40, 40, -12, 6, -2 },
487     }
488 };
489
444 ATTR_MCMODEL_SMALL
445 const int8_t ALIGN(david_mc_subpel_filters[6][15][0], 64) = {
446     [DAVID_FILTER_8TAP_REGULAR] = {
447         { 0, 1, -7, 38, 38, -7, 1, 0 },
448     }, [DAVID_FILTER_8TAP_SMOOTH] = {
449     }, [DAVID_FILTER_8TAP_SHARP] = {
450     },
451     { -2, 6, -12, 40, 40, -12, 6, -2 },
452 };

```

Рисунок 31. дельта файла tables.c

3.1.2.3. Сокращение количества фильтров до 4

В отличие от метода 1, в данном методе не получится просто опустить 2 и 7 фильтры ввиду того, что от этого потеряется баланс суммы коэффициентов, равный 128, ввиду чего будет теряться качество не только в областях интерполяции, но и на всем изображении в целом. Некоторые области будут засветляться или затемняться, интерполируемые от них области также будут менять Luma-компонент, ввиду чего ошибки будут копиться экспоненциально. В связи с этим необходимо переписать значения коэффициентов фильтров, сохраняя сумму 128. Учитывая, что в методе 2 количество вариантов интерполяции было сокращено до 6, объем работы снижен радикально. В DAV1D сумма коэффициентов равна 64, но это не меняет сути, так как в дальнейшем к коэффициентам при интерполяции применяется битовый сдвиг влево, то есть умножение на 2.

Данная оптимизация сокращает время декодирования на 33.1%, 365с со средним значением 373с.

<pre> 77 #define FILTER_8TAP(src, x, F, stride) \ 78 (\ 79- F[1] * src[x + -2 * stride] + \ 80- F[2] * src[x + -1 * stride] + \ 81- F[3] * src[x + +0 * stride] + \ 82- F[4] * src[x + +1 * stride] + \ 83- F[5] * src[x + +2 * stride] + \ 84- F[6] * src[x + +3 * stride] \ 85-) 86 87 #define FILTER_8TAP2(src, x, F) \ 88 (\ 89- F[1] * src[1][x] + \ 90- F[2] * src[2][x] + \ 91- F[3] * src[3][x] + \ 92- F[4] * src[4][x] + \ 93- F[5] * src[5][x] + \ 94- F[6] * src[6][x] \ 95-) </pre>	<pre> 77 #define FILTER_8TAP(src, x, F, stride) \ 78 (\ 79+ F[0] * src[x + -1 * stride] + \ 80+ F[1] * src[x + +0 * stride] + \ 81+ F[2] * src[x + +1 * stride] + \ 82+ F[3] * src[x + +2 * stride] \ 83-) 84 85 #define FILTER_8TAP2(src, x, F) \ 86 (\ 87+ F[0] * src[2][x] + \ 88+ F[1] * src[3][x] + \ 89+ F[2] * src[4][x] + \ 90+ F[3] * src[5][x] \ 91-) 92 </pre>
--	---

Рисунок 32. дельта файла *mc_tmpl.c*

<pre> 444 ATTR_MCMODEL_SMALL 445- const int8_t ALIGN(dav1d_mc_subpel_filters[6][15] 446- [8], 64) = { 447- [DAV1D_FILTER_8TAP_REGULAR] = { 448- { 0, 1, -7, 38, 38, -7, 1, 0 }, 449- { 0, -1, 7, 26, 26, 7, -1, 0 }, 450- { 0, -2, 6, -12, 40, 40, -12, 6, -2 }, 451- /* width <= 4 */ 452- { 3 + DAV1D_FILTER_8TAP_REGULAR } = { 453- { 0, 0, -6, 38, 38, -6, 0, 0 }, 454- { 3 + DAV1D_FILTER_8TAP_SMOOTH } = { 455- { 0, 0, 6, 26, 26, 6, 0, 0 }, 456- /* Bilin scaled being very rarely used, add a 457- * new table entry 458- * and use the put/interp_8tap_scaled code, thus 459- * acting as a 460- * scaled bilinear filter. */ 461- { 0, 0, 0, 32, 32, 0, 0, 0 }, 462- }, 463- }; 464- }; </pre>	<pre> 444 ATTR_MCMODEL_SMALL 445+ const int8_t ALIGN(dav1d_mc_subpel_filters[6][1] 446+ [8], 64) = { 447+ [DAV1D_FILTER_8TAP_REGULAR] = { 448+ { -7, 39, 39, -7 }, 449+ { 6, 26, 26, 6 }, 450+ { -12, 44, 44, -12 }, 451+ /* width <= 4 */ 452+ { 3 + DAV1D_FILTER_8TAP_REGULAR } = { 453+ { -6, 38, 38, -6 }, 454+ { 6, 26, 26, 6 }, 455+ /* Bilin scaled being very rarely used, add a 456+ * new table entry 457+ * and use the put/interp_8tap_scaled code, thus 458+ * acting as a 459+ * scaled bilinear filter. */ 460+ { 0, 32, 32, 0 }, 461+ }, 462+ }; 463- }; 464- }; </pre>
--	--

Рисунок 33. дельта файла *tables.c*

3.1.2.4. Сокращение дробного остатка

При вычислении интерполированного значения используется следующая формула:

$$(A * X[0] + B * X[1] \dots + (1 \ll 7) \gg 1) \gg 7, \quad (8)$$

Где А..Н – коэффициенты интерполяции,

X – массив интерполируемых пикселей,

В данной формуле битовый сдвиг на 7 отвечает за быстрое деление на 128 (отсюда и вытекает требование к сумме коэффициентов), а часть с $(1 \ll 7) \gg 1$ является выражением для увеличения точности, сохраняя нечетные биты. Данная оптимизация вырезает это выражение, достигая прироста скорости в 36% при времени декодирования в 349с и среднем времени 353.9с.

```

94 #define DAV1D_FILTER_8TAP_RND(src, x, F, stride, sh) \
95- ((FILTER_8TAP(src, x, F, stride) + ((1 << (sh)) >> 1)) >> (sh))
96
97 #define DAV1D_FILTER_8TAP_RND2(src, x, F, stride, rnd, sh) \
98- ((FILTER_8TAP(src, x, F, stride) + (rnd)) >> (sh))
99
100 #define DAV1D_FILTER_8TAP_RND3(src, x, F, sh) \
101- ((FILTER_8TAP2(src, x, F) + ((1 << (sh)) >> 1)) >> (sh))
102
94 #define DAV1D_FILTER_8TAP_RND(src, x, F, stride, sh) \
95+ ((FILTER_8TAP(src, x, F, stride) >> (sh))
96
97 #define DAV1D_FILTER_8TAP_RND2(src, x, F, stride, rnd, sh) \
98- ((FILTER_8TAP(src, x, F, stride) + (rnd)) >> (sh))
99
100 #define DAV1D_FILTER_8TAP_RND3(src, x, F, sh) \
101+ ((FILTER_8TAP2(src, x, F) >> (sh))
102

```

Рисунок 34. дельта файла *tc_tmpl.c*

3.1.3. Оптимизация обратных трансформаций

Как было описано в главе 2, данная оптимизация затрагивает обратные трансформации частот, убирая половину частот в каждой из DCT трансформаций. Экспериментальным путем было выявлено, что избавление от 75% частот вместо 50% приводит к сильным артефактам видео, искажающим его качество.

Данная оптимизация повышает прирост скорости до 38.1% при времени декодирования 337с и среднем времени декодирования 355с.

```

65 static NOINLINE void
66 inv_dct4_1d_internal_c(int32_t *const c, const ptrdiff_t stride,
67- const int min, const int max, const int tx64)
68 {
69- assert(stride > 0);
70
71- const int in0 = c[0 * stride], in1 = c[1 * stride];
72
73- int t0, t1, t2, t3;
74- if (tx64) {
75- t0 = t1 = (in0 * 181 + 128) >> 8;
76- t2 = (in1 * 1567 + 2048) >> 12;
77- t3 = (in1 * 3784 + 2048) >> 12;
78- } else {
79- const int in2 = c[2 * stride], in3 = c[3 * stride];
80- t0 = ((in0 + in2) * 181 + 128) >> 8;
81- t1 = ((in0 - in2) * 181 + 128) >> 8;
82- t2 = ((in1 * 1567 - in3 * (3784 - 4096) + 2048) >> 12) - in3;
83- t3 = ((in1 * (3784 - 4096) + in3 * 1567 + 2048) >> 12) + in1;
84- }
85- c[0 * stride] = CLIP(t0 + t3);
86- c[1 * stride] = CLIP(t1 + t2);
87- c[2 * stride] = CLIP(t1 - t2);
88- c[3 * stride] = CLIP(t0 - t3);
89- }
90
65 static NOINLINE void
66 inv_dct4_1d_internal_c(int32_t *const c, const ptrdiff_t stride,
67- const int min, const int max, const int tx64)
68 {
69+ assert(stride > 0);
70+ //memset(&c[(4 / XTREME_CUT) * stride], 0, (4 - (4 / XTREME_CUT)) * sizeof(int32_t));
71+
72+ const int in0 = c[0 * stride], in1 = c[1 * stride];
73+
74+ int t0, t1, t2, t3;
75+ if (tx64) {
76+ t0 = t1 = (in0 * 181 + 128) >> 8;
77+ t2 = (in1 * 1567 + 2048) >> 12;
78+ t3 = (in1 * 3784 + 2048) >> 12;
79+ } else {
80+ const int in2 = c[2 * stride], in3 = c[3 * stride];
81+ t0 = ((in0 + in2) * 181 + 128) >> 8;
82+ t1 = ((in0 - in2) * 181 + 128) >> 8;
83+ t2 = ((in1 * 1567 - in3 * (3784 - 4096) + 2048) >> 12) - in3;
84+ t3 = ((in1 * (3784 - 4096) + in3 * 1567 + 2048) >> 12) + in1;
85+ }
86+ c[0 * stride] = CLIP(t0 + t3);
87+ c[1 * stride] = CLIP(t1 + t2);
88+ c[2 * stride] = CLIP(t1 - t2);
89+ c[3 * stride] = CLIP(t0 - t3);
90+ }
91

```

Рисунок 35. дельта файла *itx_1d.c*

3.1.4. Оптимизация реконструкции изображения

В ходе разработки также был вырезан фильтр Винера, используемый для реконструкции кадра. Прирост производительности составил 38.5%, время декодирования – 335.9с при среднем времени в 339с.

```

248 // FIXME Could split into luma and chroma specific functions,
249 // (since first and last tops are always 0 for chroma)
250 static void wiener_c(pixel *p, const ptrdiff_t stride,
251- const pixel *left[4],
252- const pixel *lpf, const int w, int h,
253- const LooprestorationParams *const params,
254- const enum LiEdgeFlags edges
255- HIGHBD_DECL_SUFFIX)
256 {
257- // Values stored between horizontal and vertical filtering don't
258- // fit in a uint8_t.
259- uint16_t hor[6 * REST_UNIT_STRIDE];
260- uint16_t *ptrs[7], *rows[6];
261- for (int i = 0; i < 6; i++)
262- rows[i] = &hor[i * REST_UNIT_STRIDE];
263
248 // FIXME Could split into luma and chroma specific functions,
249 // (since first and last tops are always 0 for chroma)
250 static void wiener_c(pixel *p, const ptrdiff_t stride,
251- const pixel *left[4],
252- const pixel *lpf, const int w, int h,
253- const LooprestorationParams *const params,
254- const enum LiEdgeFlags edges
255- HIGHBD_DECL_SUFFIX)
256 {
257+ return;
258+ // Values stored between horizontal and vertical filtering don't
259+ // fit in a uint8_t.
260+ uint16_t hor[6 * REST_UNIT_STRIDE];
261+ uint16_t *ptrs[7], *rows[6];
262+ for (int i = 0; i < 6; i++)
263+ rows[i] = &hor[i * REST_UNIT_STRIDE];
264

```

Рисунок 36. дельта файла *looprestoration_tmpl.c*

3.1.5. Выводы

На рис. 37 и 38 ниже представлены время декодирования видео и прирост производительности в зависимости от оптимизаций. Оптимизации применяются последовательно, поэтому эффект от них – кумулятивный. Суммарный эффект в 38.5% ускорения декодирования отвечает поставленным нефункциональным требованиям.

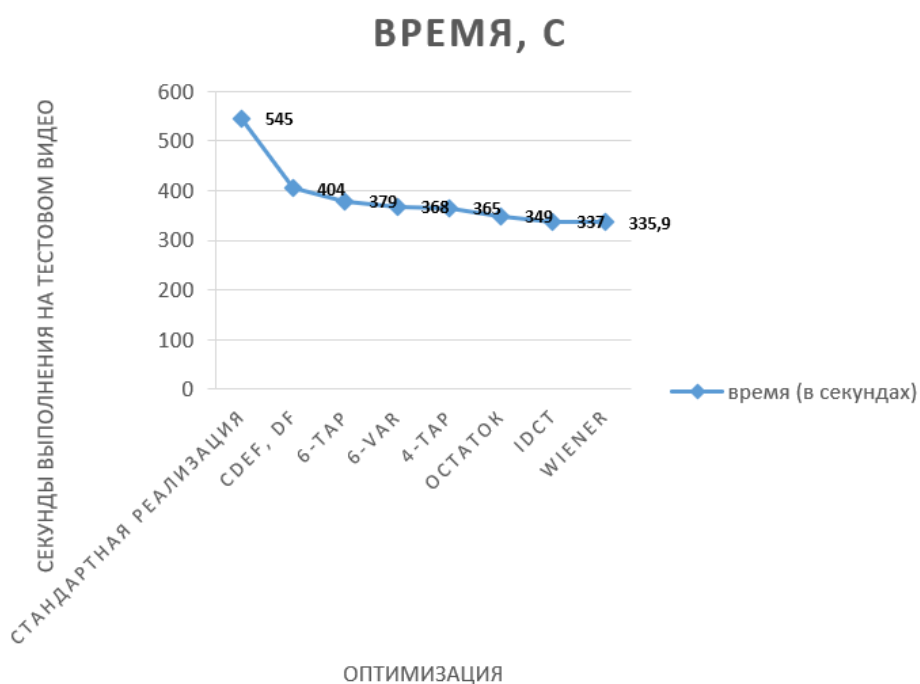


Рисунок 37. зависимость времени декодирования от оптимизаций

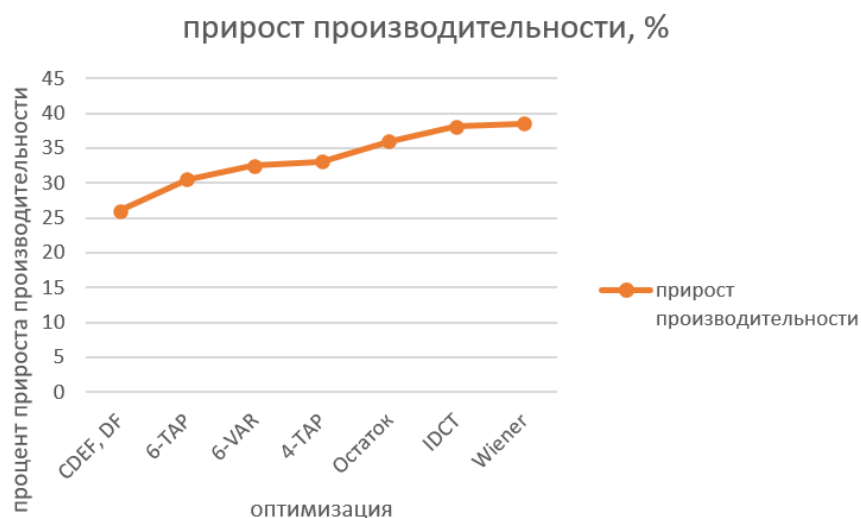


Рисунок 38. зависимость прироста скорости от оптимизаций

3.2. Финальная система

В данном разделе будут описываться 2 системы: наивная, неоптимизированная система и финальная система со всеми оптимизациями.

3.2.1. Реализация наивной системы

Для данной системы был выбран упрощённый стек из стандартного DAV1D, Fastapi и PostgreSQL. Данная система не масштабируется и медленно обрабатывает одиночные изображения.

Система состоит из трех контейнеров: фронтенд, бэкенд, совмещающий авторизацию и декодирование, а также контейнер базы данных. Бэкенд использует стандартный декодер DAV1D.

1. Реализация Backend

В backend api главной функцией является upload. Данная функция с помощью FFMPEG отделяет obu-файл (представляющий собой видеопоток av1 кодека) от всего файла, декодирует с помощью стандартного av1 первый кадр, затем с помощью него декодирует остальной поток и возвращает первый кадр. Данная функция представлена на рис. 39.

```

171 < import io
172 < from PIL import Image
173 < @app.post("/upload")
174 < async def upload_video(file: UploadFile = File(...)):
175 <     """Принимает MP4/MKV с AV1-видео, извлекает OBU-поток через ffmpeg,
176 <     декодирует первый кадр через dav1d и возвращает его в виде JPEG."""
177 <     with tempfile.TemporaryDirectory() as tmpdir:
178 <         input_container = os.path.join(tmpdir, "input." + file.filename.split('.')[-1])
179 <         with open(input_container, "wb") as f:
180 <             while chunk := await file.read(8192):
181 <                 f.write(chunk)
182 <         obu_path = os.path.join(tmpdir, "stream.obu")
183 <         result = subprocess.run(
184 <             [FFMPEG_PATH, "-i", input_container, "-c", "copy", "-f", "av1", obu_path],
185 <             capture_output=True,
186 <         )
187 <         if result.returncode != 0:
188 <             raise HTTPException(
189 <                 status_code=422,
190 <                 detail=f"ffmpeg conversion to OBU failed: {result.stderr.decode()}"
191 <             )
192 <         result = subprocess.run(
193 <             [DAV1D_PATH, "-i", obu_path, "-o", "/dev/null"],
194 <             capture_output=True,
195 <         )
196 <         if result.returncode != 0:
197 <             raise HTTPException(
198 <                 status_code=422,
199 <                 detail=f"dav1d decode failed: {result.stderr.decode()}",
200 <             )
201 <         frame_y4m = os.path.join(tmpdir, "frame.y4m")
202 <         result = subprocess.run(
203 <             [DAV1D_PATH, "-i", obu_path, "-o", frame_y4m, "--threads", "1", "--limit", "1"],
204 <             capture_output=True,
205 <         )
206 <         if result.returncode != 0:
207 <             raise HTTPException(
208 <                 status_code=422,
209 <                 detail=f"dav1d first-frame decode failed: {result.stderr.decode()}",
210 <             )
211 <         frame_bytes = _y4m_to_jpeg(frame_y4m)
212 <         return Response(content=frame_bytes, media_type="image/jpeg")

```

Рисунок 39. naïve upload

Функция также полагается на авторизацию, которая происходит на каждом запросе как обращение к реляционной базе данных ввиду «наивности» реализации. Сервис авторизации использует паттерн access/refresh, для пользователя создаётся короткий access token и долго живущий refresh token. Хэширование происходит по алгоритму SHA256 ввиду того, что Comexr нацелена на западный рынок, ввиду чего не имеет смысла использовать российские алгоритмы шифрования.

Frontend реализован как простое React приложение с двумя экранами.

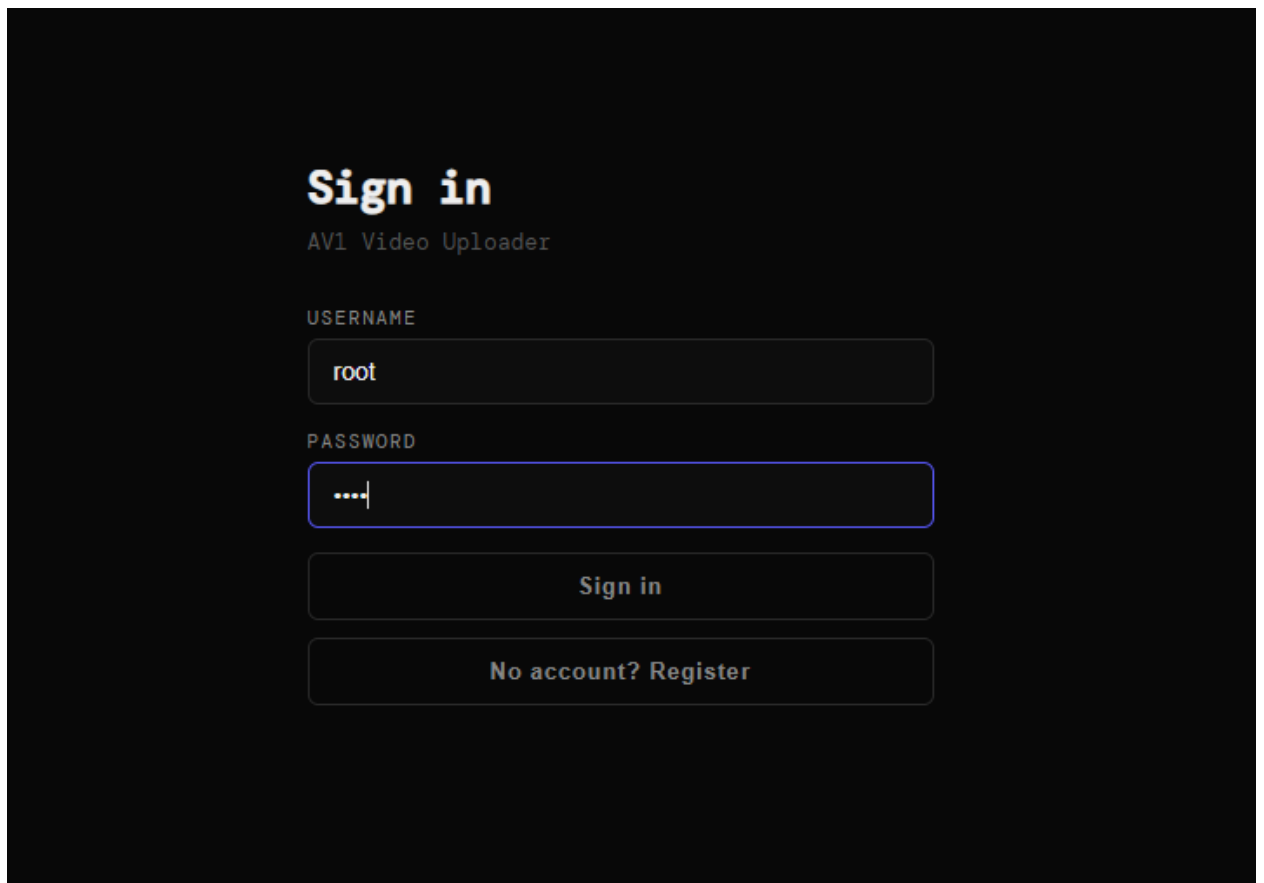


Рисунок 40. экран входа

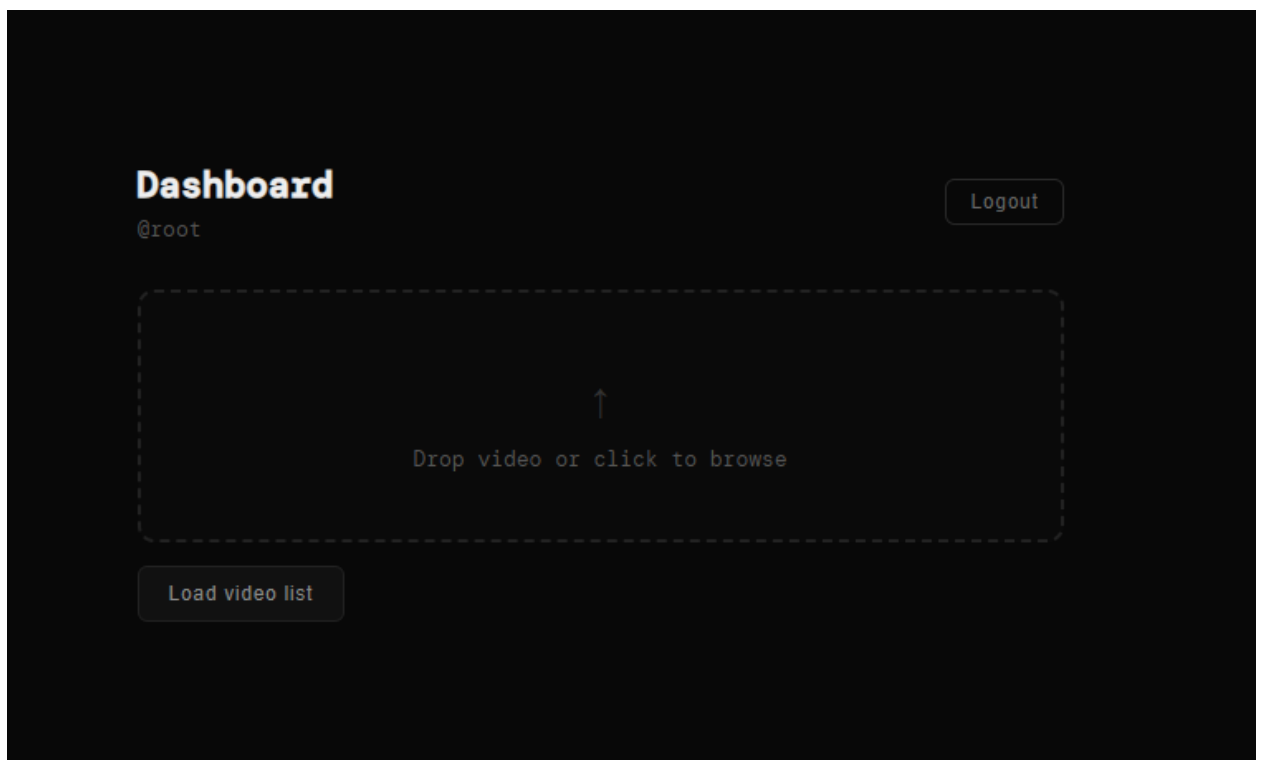


Рисунок 41. экран загрузки

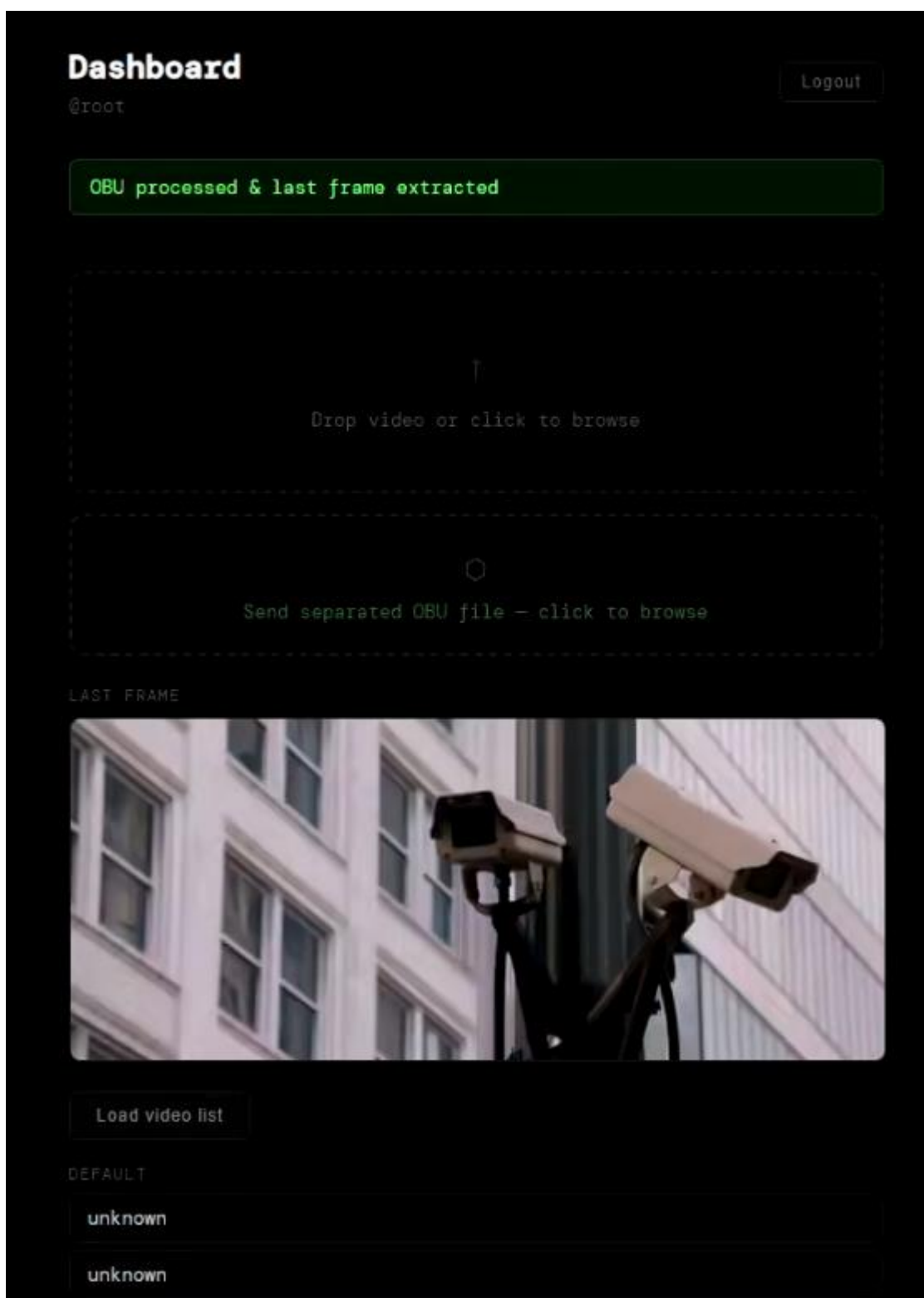


Рисунок 42. Экран с загруженным файлом

3.2.2. Реализация оптимизированной системы

Оптимизированная система в целом похожа на наивную реализацию, но имеет больше деталей и оптимизаций.

3.2.2.1. Backend

Самая очевидная оптимизация – использование оптимизированного DAV1D в качестве видеodeкодера.

Backend состоит из 6 контейнеров, ниже – пояснения по каждому из них.

- Common-api – API авторизации, необходимое для бизнес-функций.
- Gateway – балансировщик и прокси, проксирует все запросы кроме связанных с upload на common-api, а запросы связанные с upload – на наименее занятую реплику в соответствии с алгоритмом, описанным в разделе 2.2.2. Аккумулирует состояния Decoder-api.
- Decoder-api – API декодирования, использующее оптимизированную версию DAV1D для декодирования видео. Отправляет свой статус на Gateway.
- Nats – контейнер с очередью NATS Jetstream.
- Redis – контейнер с Redis, хранящий горячие данные.
- Db – контейнер с Postgresql, хранящий реляционные данные.

Рассмотрим подробнее каждый первые три.

3.2.2.2. Common-api

Данный контейнер во многом похож на вышеописанный контейнер common-api наивной реализации.

Данный контейнер реализует все необходимые эндпоинты, описанные во второй главе.

Эндпоинт /user, необходимый для авторизации пользователя, устроен по принципу процессорного кеша, где L1 выступает Redis, а L3 – PostgreSQL. При получении запроса на получение данных сначала проверяется access token в Redis, а в случае ошибки происходит «холодный промах», при котором проверяется значение в PostgreSQL и в случае успеха сохраняется в Redis.

Access и refresh токены хранятся в реляционной базе данных.

3.2.2.3. Gateway

Как уже было описано во второй главе, проблемой становится уведомление пользователя о конце декодирования. Для решения этой проблемы был применен описанный подход: Gateway открывает вебсокет-соединение по адресу

/ws/client_notification, к которому подключаются клиенты передавая свой access token. Далее клиент добавляется в список рассылки, где ключом служит id пользователя, Gateway входит в блокирующее чтение jetstream очереди notifications и при появлении сообщения, адресованного пользователю, отправляет его. В случае если вебсокет закрыт на стороне клиента данный клиент удаляется из списка рассылки. На рис. 43 представлена функция, отвечающая за рассылку.

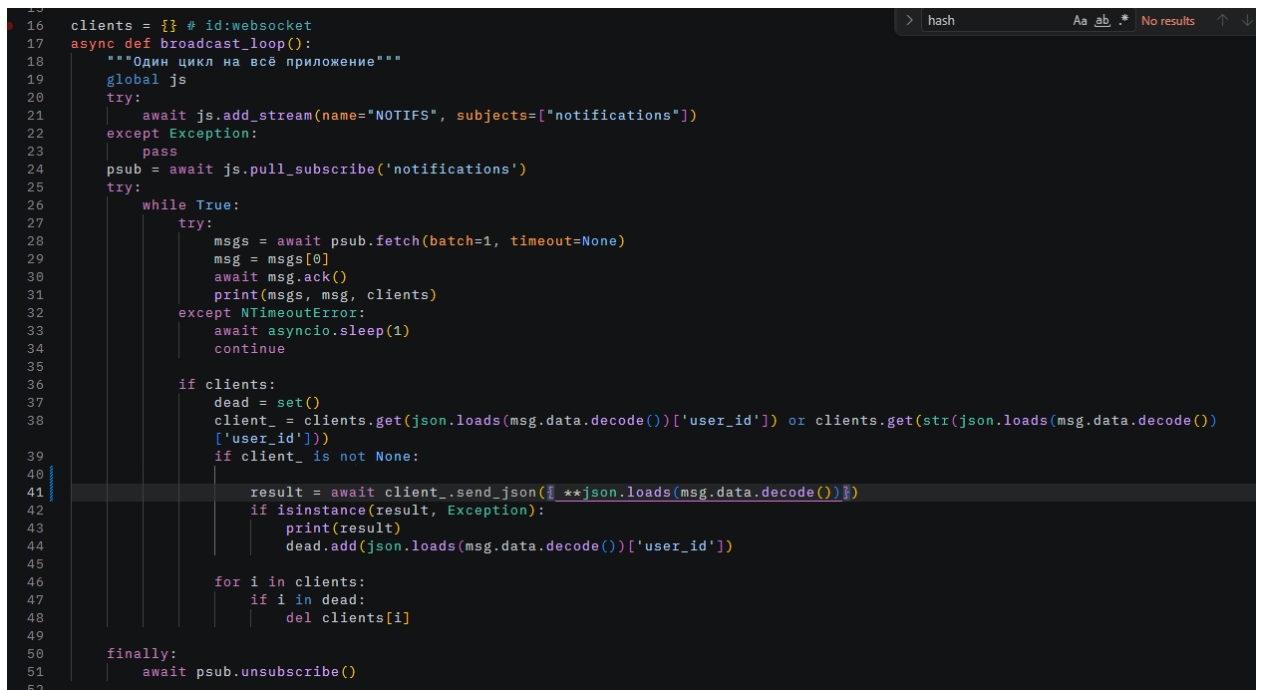
The image shows a code editor with a dark background. The code is written in Python and defines a function named 'broadcast_loop'. It starts with a dictionary 'clients' and an 'async def' definition. Inside, there's a 'global js' statement, followed by a 'try' block for adding a stream and a 'pass' statement. Then, 'psub' is assigned to 'await js.pull_subscribe('notifications')'. A 'while True' loop follows, containing a 'try' block for fetching messages, acknowledging them, and printing. It also has an 'except NTimeoutError' block with an 'await asyncio.sleep(1)' and 'continue'. After the loop, there's an 'if clients' block that updates a 'dead' set, checks for client presence, sends JSON data, and updates the 'dead' set. A 'for' loop then iterates over 'clients' to remove dead entries. Finally, a 'finally' block calls 'await psub.unsubscribe()'. The code is numbered from 16 to 52 on the left margin. The editor's top bar shows 'hash', 'Aa', 'ab', and 'No results'.

Рисунок 43. функция рассылки сообщений

Второй проблемой является масштабирование нагрузки, для которой был спроектирован алгоритм балансировки. Реплики decoder-api сортируются в первую очередь по загруженности процессора, а во вторую – по оперативной памяти. Разумеется, редко два процессора загружены на идентичное количество работы, так что память будет учитываться редко. Тем не менее, в алгоритм маршрутизации также заложен критерий выбора первого сервера с достаточным количеством оперативной памяти для данного видео. Это необходимо для того, чтобы максимально быстро декодировать видео прямо из памяти, вместе с тем предотвращая заполнение памяти реплики более чем на 90%. В случае, если памяти нет ни на одном сервере, прокси ищет сервер с необходимым количеством дисковой памяти. В случае, если нет и его, возвращается сообщение о том, что свободных реплик нет.

Также Gateway получает информацию от серверов-реплик через вебсокеты по адресу /ws/replica с информацией о загрузке ЦПУ за последнюю минуту, свободной оперативной памяти и свободной дисковой памяти.

Чтобы уменьшить сетевые накладные расходы было принято решение отказаться от прямого проксирования к Decoder-api, потому что в таком случае появились бы издержки на передачу данных к одному дополнительному серверу. Вместо этого при GET запросе к API маршрутизации с access токеном Gateway формирует одноразовый токен, в котором закодирован user и выдаёт ссылку на конкретную реплику декодера с встроенным в неё вышеупомянутым токеном.

3.2.2.4. Decoder-api

Decoder-api, будучи API для декодирования, имеет основную функцию upload, принимающую на вход OBU-файл, представляющий собой видеопоток кодака AV1, сохраняющий в оперативную память для уменьшения задержек чтения данных, а после этого декодирующий с помощью оптимизированного видеodeкодера DAV1D. После декодирования посылается сообщение в Jetstream очередь с полезной нагрузкой и адресом получателя.

3.2.2.5. Frontend

Frontend во многом напоминает наивную реализацию, однако для сокращения сетевых издержек был разработан алгоритм вырезания из видео аудио. Для этого был скомпилирован FFmpeg в целевую платформу WASM с помощью Emscripten версии 4.0.12, затем интегрирован в React приложение.

3.3. Тестирование

В главе 2 были определены следующие метрики: общая задержка от начала отправки видео на сервер до полного декодирования видео в секундах; минимальное время декодирования видео без учета времени передачи видео по результатам пяти тестовых запусков; покадровая метрика потери качества видео, вычисляемая как разница Luma значений каждого пикселя кадра видео; покадровая метрика PSNR; время ответа при нагрузочном тестировании системы. Для каждого из критериев будет дано подробное объяснение релевантности критерия, протокол тестирования и его результат.

Для тестирования обе системы были развернуты на 1-2 серверах Timeweb Cloud с минимальной конфигурацией: 15Гб дискового пространства, 1Гб оперативной памяти, 1 ядро процессора.

3.3.1. Общая задержка от начала отправки видео на сервер до полного декодирования

Данный критерий проверяет систему с позиции пользователя: насколько для него изменилось взаимодействие с системой через frontend.

Для данной метрики был написан скрипт, находящийся в репозитории проекта по пути `benchmarks\throughput\bench.py`. Данный скрипт последовательно загружает OBU-видео в оптимизированную систему и MKV-видео в наивную. Загрузка WASM модуля не эмулируется, так как в оптимизированной системе после первой загрузки он кешируется.

В качестве результата выбирается минимальное время декодирования, так как, по словам профессора MIT Чарльза Лейзерсона, минимальное время выполнения показывает время выполнения с минимальной зашумлённостью [26].

По результатам тестирования минимальное время для оптимизированной системы: 165.9 секунд, для наивной: 283.4 секунд. Бенчмарки для обеих систем выполнялись одновременно и параллельно, чтобы исключить задержки, связанные с флуктуацией скорости сети.

3.3.2. Минимальное время декодирования видео без учета времени передачи

Данная метрика уже была описана в разделе 3.1, для её замера был разработан скрипт, находящийся в репозитории проекта по пути `benchmarks/decoder/dav1d.sh`. В ходе профилирования было выявлено, что флуктуации скорости декодирования – незначительны ввиду того, что декодируется полчасовое видео. В связи с этим, а также для сокращения времени всего процесса и, соответственно, увеличения скорости разработки, было принято решение измерять производительность по трём итерациям.

3.3.3. Покадровая метрика потери качества видео

Данная метрика не является стандартом индустрии и необходима скорее для внутренней относительной оценки декодера в компании Comexp. Для её оценки был реализован скрипт, находящийся по пути `benchmarks\quality\framewise_luma.py`.

Скрипт вычисляет покадровое отклонение каждого пикселя декодированного видео с помощью оптимизированного декодера и видео декодированного с помощью оригинального декодера, затем вычисляет среднее по кадру и среднее по видео. Результаты бенчмарка представлены в файле-отчёте по пути `benchmarks\quality\framewise1000.md`. Средняя мера отклонения на тестовом видео при вычислении по тысяче фреймов (примерно 40 секунд) – 6.28 с максимумом 9.62. Данная метрика может принимать значение от 0 до 255, получившийся результат был признан Comexr как приемлемый. На рис. 44 представлен график среднего отклонения по фрейму (сверху) и кумулятивный график отклонения (снизу). Как можно увидеть, на верхнем графике имеются резкие спады – предположительно это связано с тем, что при декодировании Golden frame сбрасывается кеш предсказания и накопленные ошибки вместе с ним.

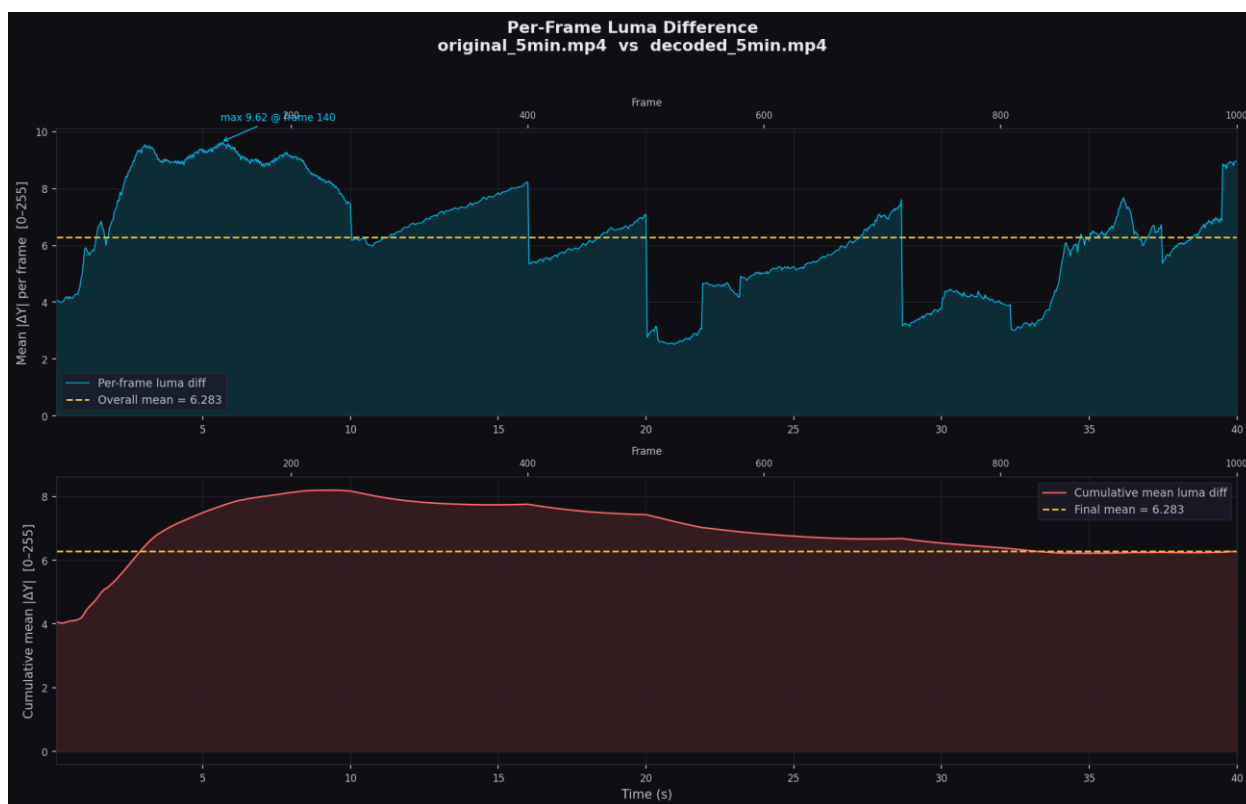


Рисунок 44. график покадрового отклонения luma

3.3.4. Покадровая метрика PSNR

Следующая метрика – Peak Signal-to-Noise Ratio. В отличие от `framewise luma`, данная метрика широко используется в видеоиндустрии для оценки деградации качества видео по сравнению с оригиналом. Для её вычисления был разработан скрипт, находящийся по пути `benchmarks\quality\psnr.py`, отчёт о выполнении представлен в `benchmarks\quality\psnr1000frames.md`. По результатам выполнения

бенчмарка на 1000 фреймов был получен результат 26.2 дБ с минимумом 21.02дБ и максимумом 35.38дБ. Это низкое значение, но оно удовлетворяет требованиям задачи.

3.3.5. Нагрузочное тестирование

Для нагрузочного тестирования был разработан скрипт, представленный по пути benchmarks\throughput\bench.py. Данный скрипт параллельно отправляет 50 видео по 38мб продолжительностью 60 секунд с задержкой 0.5с на видео. В результате тестирования оптимизированной и наивной систем были получены следующие результаты:

Таблица 6 – нагрузочное тестирование

система	Количество обработанных видео	Суммарное время выполнения бенчмарка, в секундах	Суммарный размер обработанных видео, Мб	Среднее значение скорости обработки видео, Мб/с
оптимизированная	17	150.61	646	4.28
наивная	50	957.84	1900	1.98

Как можно увидеть, оптимизированная система имеет пропускную способность более чем в 2 раза большую, нежели наивная, будучи развернутой на двух серверах. Следует отметить, что 17 обработанных видео – не ошибка, скрипт тестирования реализован так, что при получении сообщения о недостатке свободных реплик он не пытается повторить запрос.

3.4. Исходный код

Исходный код оптимизированного видеodeкодера доступен по адресу <https://github.com/ressiwave/DAV2D>. Исходный код WASM-версии FFmpeg доступен по адресу <https://github.com/ressiwave/FFFMPEG>. Исходный код всей системы доступен по адресу <https://github.com/ressiwave/UFAV1VDS>. Протокол бенчмарков видеodeкодеров доступен по адресу <https://github.com/ressiwave/AV1-bench>.

ЗАКЛЮЧЕНИЕ

В данной работе были полностью решены все поставленные задачи, а результаты удовлетворяют метрикам. При достаточном количестве серверов система может обработать любую нагрузку, а её масштабирование происходит линейно. Видеодекодер DAV1D оптимизирован на 38.5%, система целиком – более чем в 2 раза.

В результате анализа был проведен разбор необходимых частей стандарта AV1 для выявления узких мест, необходимых для работы, в частности – DF, CDEF, IDCT, интерполяция, в дальнейшем полученные знания были применены для оптимизации в главе 3. Также были сформулированы требования к работе, проведён анализ существующих решений.

В результате проектирования была разработана масштабируемая архитектура двух систем декодирования, масштабируемой оптимизируемой и немасштабируемой наивной. Также были намечены предстоящие оптимизации DAV1D.

В результате реализации и тестирования была разработана система и были получены необходимые метрики. Был оптимизирован декодер DAV1D с использованием спроектированных оптимизаций, затем встроен в оптимизированную систему. Системы были развернуты на двух серверах через Docker swarm, затем было проведено нагрузочное тестирование.

В дальнейшем предполагается доработка решения, включающая в себя подключение других оптимизированных декодеров, таких как H.264, VP9, H.265, а также масштабирование системы путем добавления серверов и миграция видеодекодеров на другие архитектуры.

ГЛОССАРИЙ

Motion Vectors (MV) – метод сжатия видео путем извлечения из видео векторов движения.

Constrained directional enhanced filter (CDEF) – метод постобработки видео для устранения артефактов.

Deblocking filter (DF) – метод постобработки видео для уменьшения блочности.

Оверхед – накладные расходы какой-либо технологии.

Видеокодек (кодек) – стандартизированный метод кодирования и реализованный видео.

Видеодекодер (декодер) – стандартизированный метод декодирования видео, реализованный в виде программного обеспечения.

TAPe – запатентованная система компании «ООО КОМЭКСП», предназначенная для индексации видео.

AOMedia Video 1 (AV1) – открытый стандарт видеокодека, разработанный консорциумом AOMedia Alliance for Open Media.

Errata (AV1-v1.0.0errata) – название стандарта, определяющего поведение видеокодека AV1.

Frame – кадр в декодируемом видео, представляющий собой последовательность пикселей. Не обязательно присутствует в конечном видео после декодирования.

Макроблок – сущность, объединяющая квадратную область из нескольких пикселей. Изначально термин из стандарта ITU-T H.264, но также будет использоваться для обозначения суперблоков из стандарта Errata.

Slice – сущность, объединяющая макроблоки.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Alliance for Open Media. AV1 Bitstream & Decoding Process Specification / Version 1.0.0 with Errata 1 [Электронный ресурс]: технический стандарт / консорциум Alliance for Open Media. – 2021. – URL: <https://aomediacodec.github.io/av1-spec/av1-spec.pdf> (дата обращения: 19.12.2025).
2. Международный союз электросвязи. Сектор стандартизации (ITU-T). *Рекомендация ITU-T H.264 (V15). Усовершенствованное кодирование изображений для общих аудиовизуальных услуг = Advanced video coding for generic audiovisual services* [Электронный ресурс]. — изд. от 13.08.2024. — (Серия H: Аудиовизуальные и мультимедийные системы; инфраструктура аудиовизуальных услуг; кодирование движущегося видео). — URL: <https://www.itu.int/rec/T-REC-H.264> (дата обращения: 19.12.2025).
3. Chen Y. An Overview of Core Coding Tools in the AV1 Video Codec / Yue Chen, D. Murherjee, J. Han // IEEE 2018 Picture Coding Symposium (PCS). — P. 41-45 – 2018. — DOI: 10.1109/PCS.2018.8456249.
4. Petreski D. Next Generation Video Compression Standards – Performance Overview / D. Petreski, T. Kartalov // IEEE International Conference on Systems, Signals and Image Processing (IWSSIP) — 2023. — P. 1–5. — DOI: 10.1109/IWSSIP58668.2023.10180261.
5. Kolodziejcki W. Speeding Up the AV1 Global Warped Motion Compensation / W. Kolodziejcki, R. Domanski, L. Agostini // 2024 IEEE 15th Latin America Symposium on Circuits and Systems (LASCAS). — 2024. — P. 1-5. — DOI: 10.1109/LASCAS60203.2024.10506160.
6. Yi Y. High-Speed CAVLC Encoder for 1080p 60-Hz H.264 Codec / Y. Yi, B. C. Song // IEEE Signal Processing Letters — vol. 15 — 2008. — P. 891–894. — DOI: 10.1109/LSP.2008.2001982.
7. NATS JetStream vs RabbitMQ vs Apache Kafka on VPS in 2025: Throughput/Latency Benchmarks, Exactly-Once vs At-Least-Once, Durability/Replication, TLS/mTLS, and the Best Message Broker for Your Stack / блог компании Onidel [Электронный ресурс]. 8.11.2025. URL: <https://onidel.com/blog/nats-jetstream-rabbitmq-kafka-2025-benchmarks> (дата обращения: 13.02.2026).
8. Angular vs. React vs. Vue.js in 2025 / блог компании DLT Software [Электронный ресурс]. 20.11.2025. URL: <https://medium.com/@dltsoft/angular-vs-react-vs-vue-js-in-2025-f24b56ff0064> (дата обращения: 13.02.2026).
9. H.264 is Magic / Sid Bala [Электронный ресурс]. 2.11.2016. URL: <https://sidbala.com/h-264-is-magic/> (дата обращения: 13.02.2026).
10. Lin Y., Efficient AV1 Video Coding Using a Multi-layer Framework / Wei-Ting Lin, Zoe Liu, Debargha Mukherjee // IEEE 2018 Data Compression Conference. — P. 368-369 – 2018. — DOI: 10.1109/DCC.2018.00045.
11. SVT-AV1 documentation: ALTREF and Overlay Pictures [Электронный ресурс]. URL: <https://gitlab.com/AOMediaCodec/SVT-AV1/-/blob/master/Docs/Appendix-Alt-Refs.md> (дата обращения: 13.02.2026).

12. Salonen N. A Comparative Study of Kafka and NATS : [дипломная работа] / N. Salonen. — Turku : University of Turku, 2025. — 62 с. — URL: https://www.utupub.fi/bitstream/handle/10024/182402/Salonen_Nino_opinnayte.pdf (дата обращения: 14.03.2026).
13. Comparing NATS, NATS JetStream, and Kafka Performance for Varying Payload Sizes / Nathan B. Crocker [Электронный ресурс]. 24.09.2024. URL: <https://medium.com/@nathanbcrocker/comparing-nats-nats-jetstream-and-kafka-performance-for-varying-payload-sizes-3538c94ac56c> (дата обращения: 14.03.2026).
14. Comparison of NATS, RabbitMQ, NSQ, and Kafka / блог компании Gcore [Электронный ресурс]. 20.06.2024. URL: <https://gcore.com/learning/nats-rabbitmq-nsq-kafka-comparison> (дата обращения: 14.03.2026).
15. Python Flask vs FastAPI vs Django: Framework Comparison 2026 / блог [Электронный ресурс]. 07.02.2026. URL: <https://dasroot.net/posts/2026/02/python-flask-fastapi-django-framework-comparison-2026/> (дата обращения: 14.03.2026).
16. Using H.264 video compression in IP video surveillance systems / блог [Электронный ресурс]. 03.04.2009. URL: <https://www.networkwebcams.co.uk/blog/h264-video-compression-in-ip-video-surveillance-systems/> (дата обращения: 23.03.2026).
17. Международный союз электросвязи. Сектор стандартизации (ITU-T). *Рекомендация ITU-T H.264 (V15). Усовершенствованное кодирование изображений для общих аудиовизуальных услуг = Advanced video coding for generic audiovisual services* [Электронный ресурс]. — изд. от 13.08.2024. — (Серия H: Аудиовизуальные и мультимедийные системы; инфраструктура аудиовизуальных услуг; кодирование движущегося видео). — р. 126. — URL: <https://www.itu.int/rec/T-REC-H.264> (дата обращения: 19.12.2025).
18. Международный союз электросвязи. Сектор стандартизации (ITU-T). *Рекомендация ITU-T H.264 (V15). Усовершенствованное кодирование изображений для общих аудиовизуальных услуг = Advanced video coding for generic audiovisual services* [Электронный ресурс]. — изд. от 13.08.2024. — (Серия H: Аудиовизуальные и мультимедийные системы; инфраструктура аудиовизуальных услуг; кодирование движущегося видео). — р. 101. — URL: <https://www.itu.int/rec/T-REC-H.264> (дата обращения: 19.12.2025).
19. Prediction of Transformed (DCT) Video Coding Residual for Video Compression / статья [Электронный ресурс]. 04.2014. — р. 6 — URL: https://www.researchgate.net/publication/261700797_Prediction_of_Transformed_DCT_Video_Coding_Residual_for_Video_Compression (дата обращения: 23.03.2026).
20. Ren J. Algorithmic Optimization for H.264 Deblocking Filter on Portable Devices / J. Ren, N. Kehtarnavaz // 2007 IEEE International Symposium on Consumer Electronics. — р. 1-6 – 2007. — DOI: 10.1109/ISCE.2007.4382168
21. Han J. A Technical Overview of AV1 / J. Han, B. Li, D. Mukherjee // Proceedings of the IEEE. — vol. 109, no. 9, p. 1435-1462. — DOI: 10.1109/JPROC.2021.3058584

22. Yi Y. High-Speed CAVLC Encoder for 1080p 60-Hz H.264 Codec / Y. Yi, B. Song // IEEE Signal Processing Letters. — vol. 15, p. 891-894, 2008. — DOI: 10.1109/LSP.2008.2001982
23. Moroz L. Efficient Floating-Point Division for Digital Signal Processing Application [Tips & Tricks] / L. Moroz, V. Samotyy // IEEE Signal Processing Magazine. — vol. 36, no. 1 — P. 159-163 – 2019. — DOI: 10.1109/MSP.2018.2875977
24. Time scale of system latencies / блог [Электронный ресурс]. 05.2014. — URL: <https://perfsolutions.blogspot.com/2014/05/time-scale-of-system-latencies.html> (дата обращения: 23.03.2026).
25. AV1 benchmark / репозиторий [Электронный ресурс]. 23.03.2026. — URL: <https://github.com/ressiwave/AV1-bench> (дата обращения: 23.03.2026).
26. 10. Measurement and Timing / курс MIT 6.172, видеолекции [Электронный ресурс]. 23.09.2019. URL: <https://www.youtube.com/watch?v=LvX3g45ynu8&list=PLUl4u3cNGP63VIBQVWguXxZZi0566y7Wf&index=10> (дата обращения: 17.05.2026).
27. 4. Measurement and Timing / курс MIT 6.172, видеолекции [Электронный ресурс]. 23.09.2019. URL: https://youtu.be/L1ung0wil9Y?si=Y7x0qU8wag98p_iA&t=4032 (дата обращения: 17.05.2026).

ПРИЛОЖЕНИЕ А. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Техническое задания разработано на основе выполненного анализа по ГОСТ 19.201-78–2010 и загружено в SmartLMS отдельно как документ «РИС-22-2-Берсенёв_ИИ_ТЗ.docx».